

Arab Academy for Science , Technology & Maritime Transport
College of Computing and Information Technology



Data Structures and Algorithms (CS212)

Section 06 : Trees



Section Index

- ✓ What is Trees ?
- ✓ Tree Terminologies & Properties
- ✓ What is a Binary Tree ?
- ✓ What is a Binary Search Tree ?
- ✓ Insert Node in a BST
- ✓ Search for a number in a BST
- ✓ Count Elements in the BST
- ✓ Find the Min in the BST

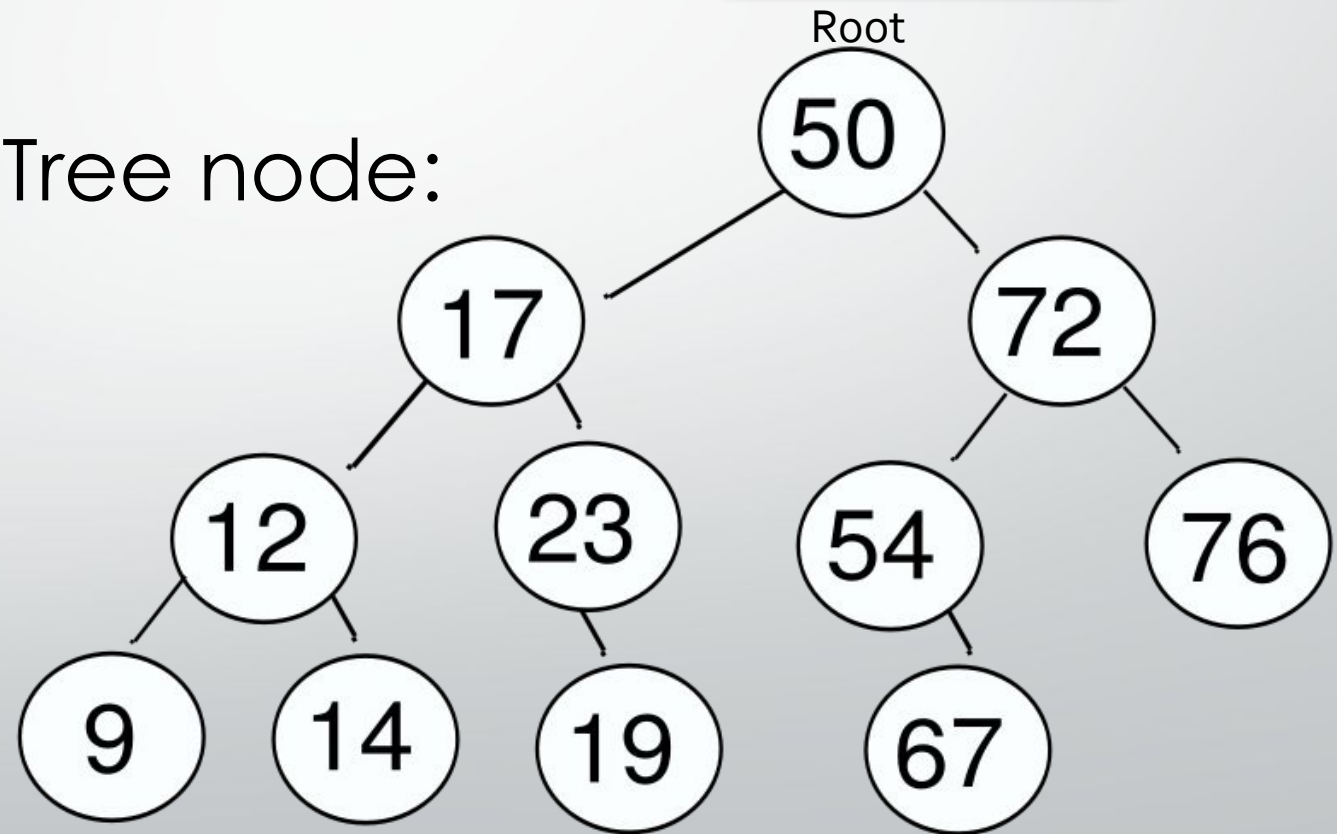
Trees



- ✓ unlike linked lists , stacks & queues ,
A Tree is **non-linear data structure**

- ✓ Structure for Binary Tree node:

```
struct Node {  
    int data ;  
    Node *left;  
    Node *right;  
};
```

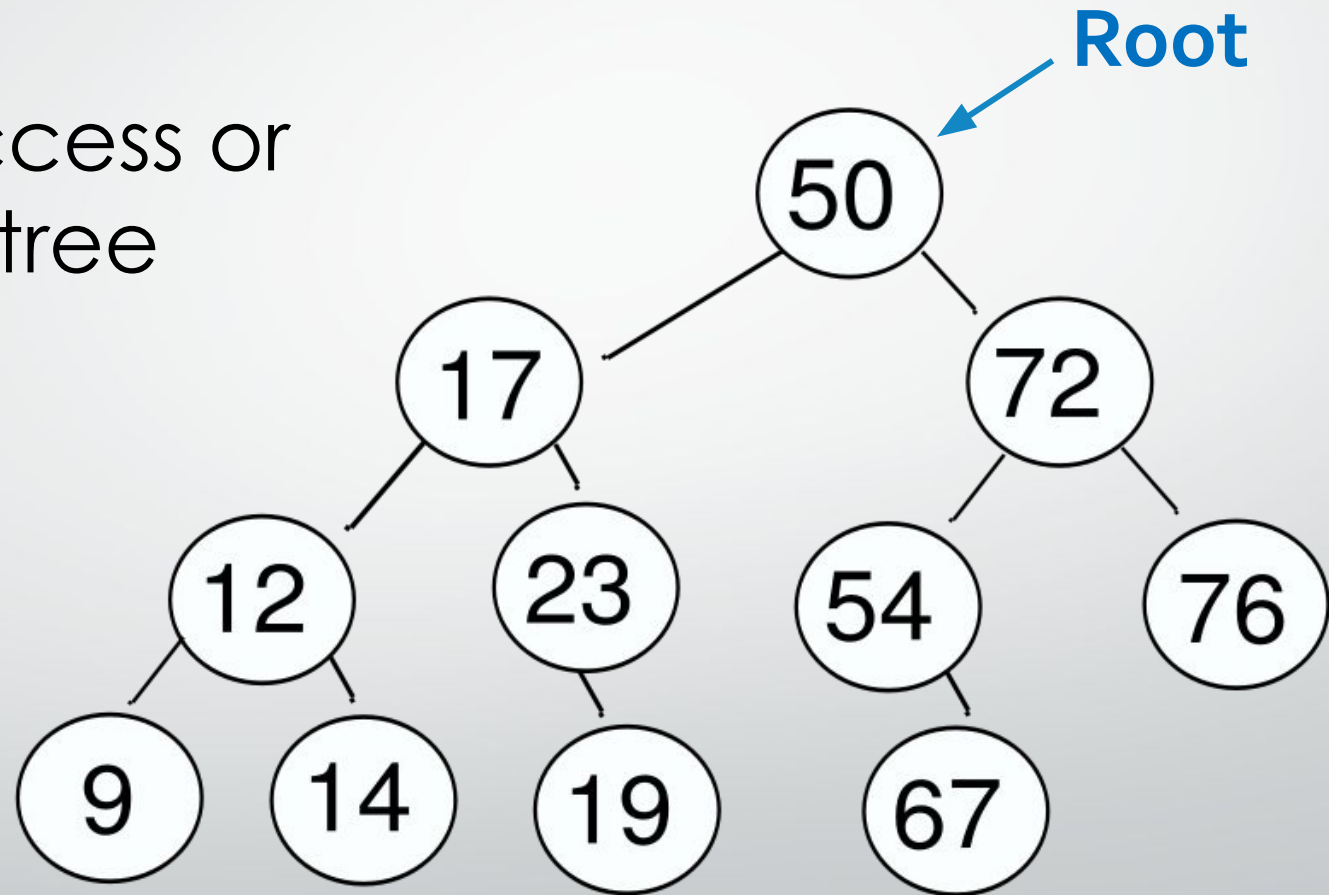


Trees: Some Terminologies

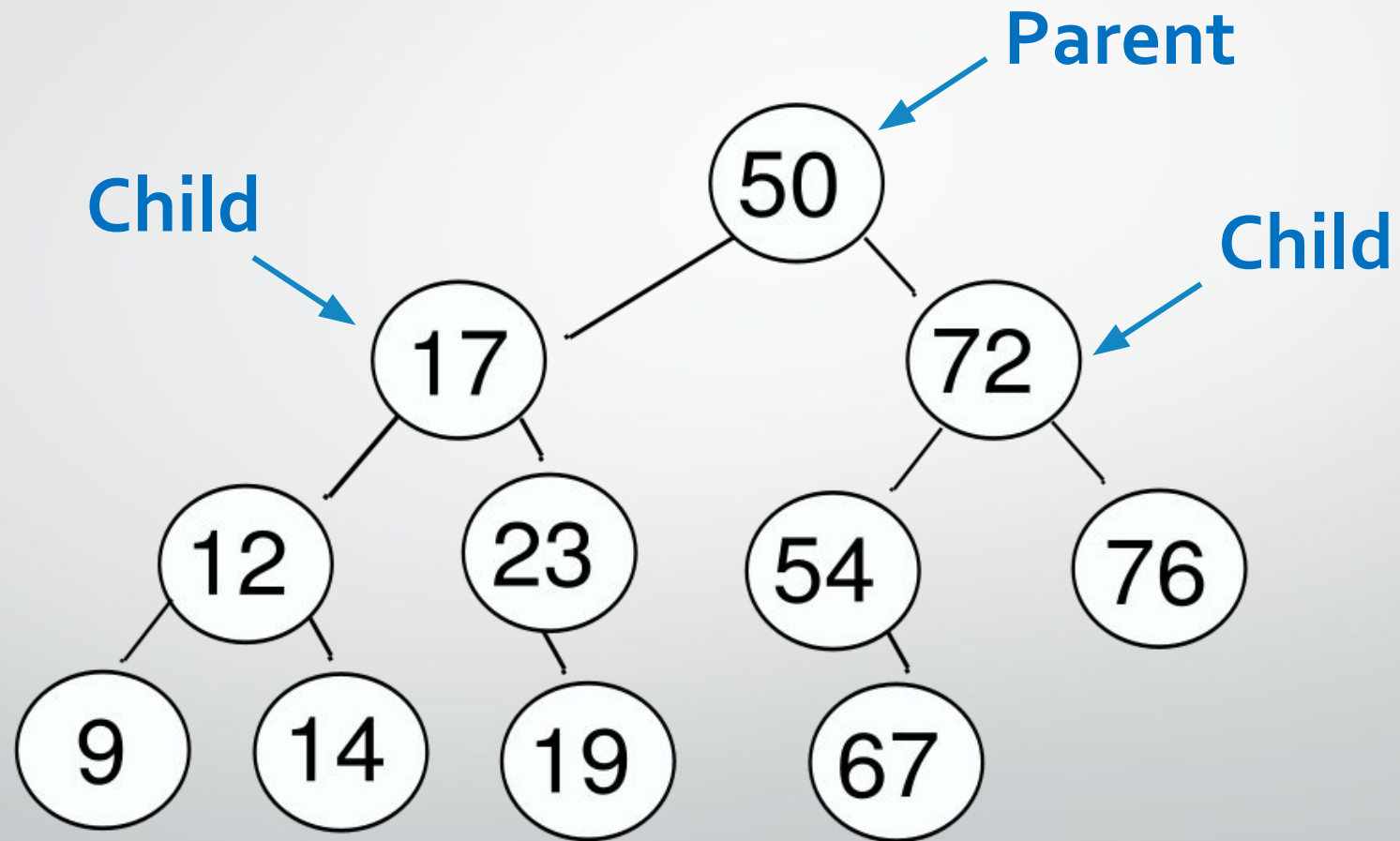
✓ **Root:**

We use it to access or traverse in our tree

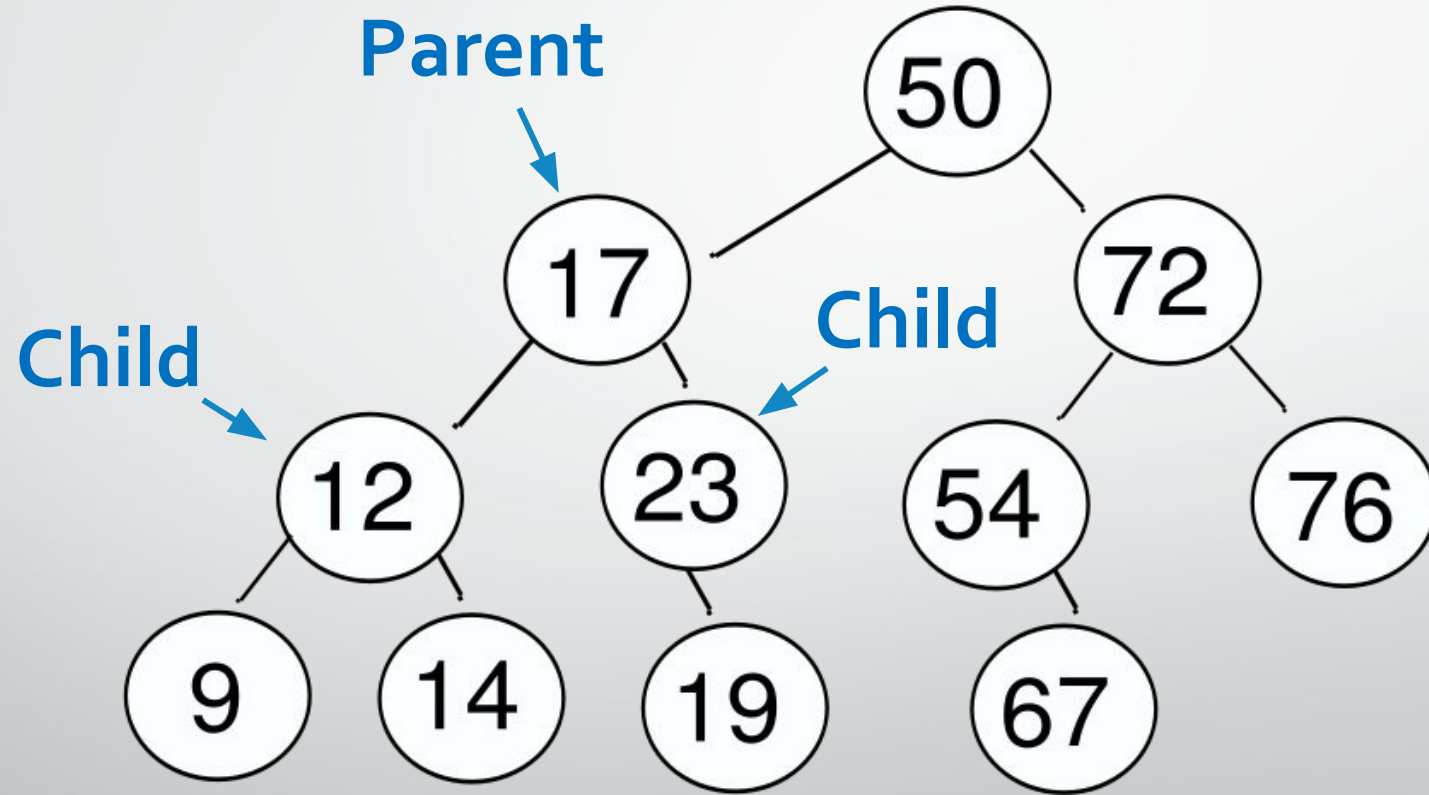
It's the identity of the Tree



Trees: Some Terminologies

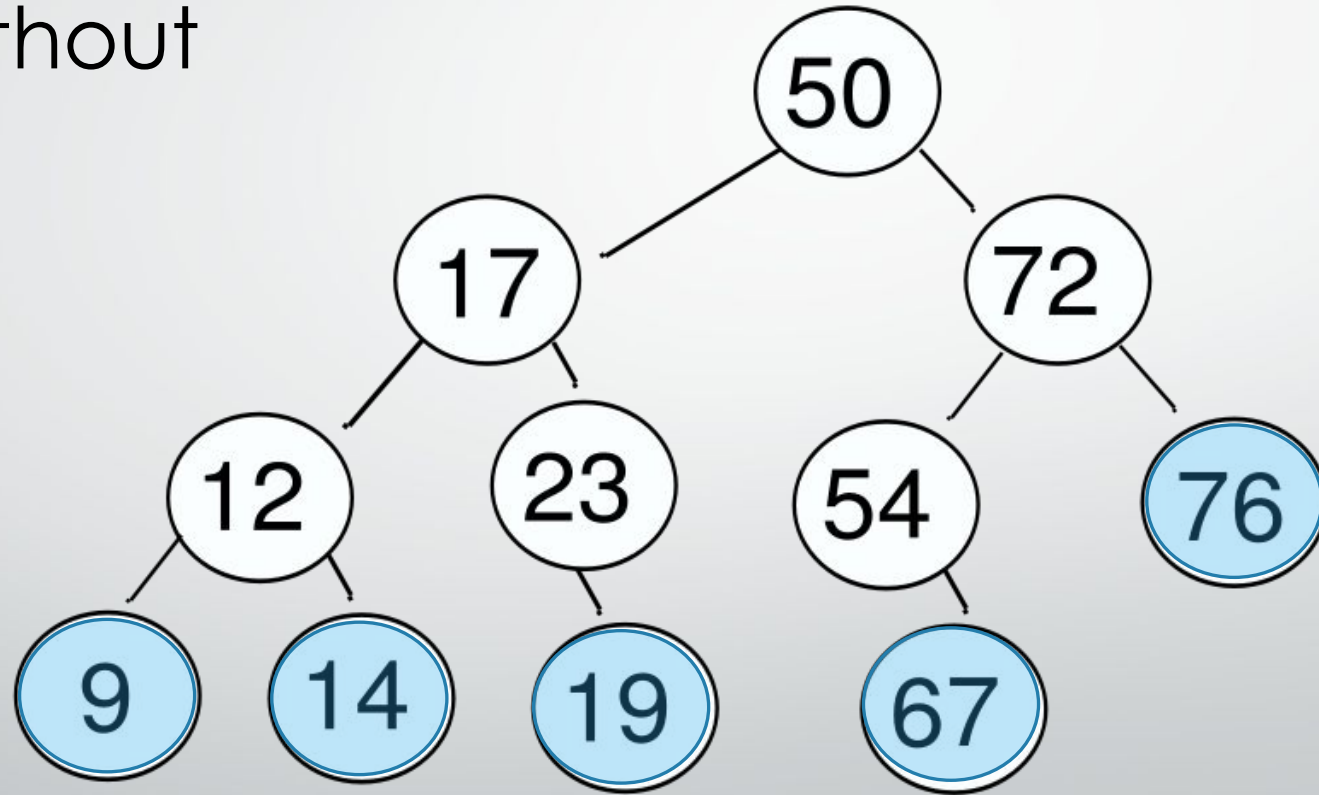


Trees: Some Terminologies



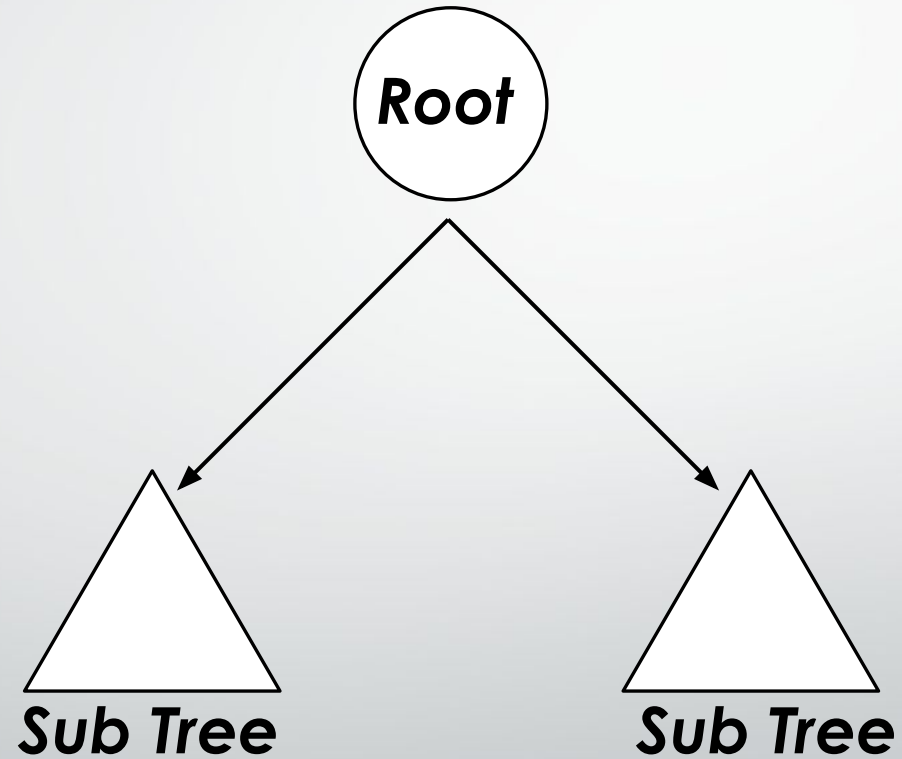
Trees: Some Terminologies

- ✓ **Leaf:**
Node without children



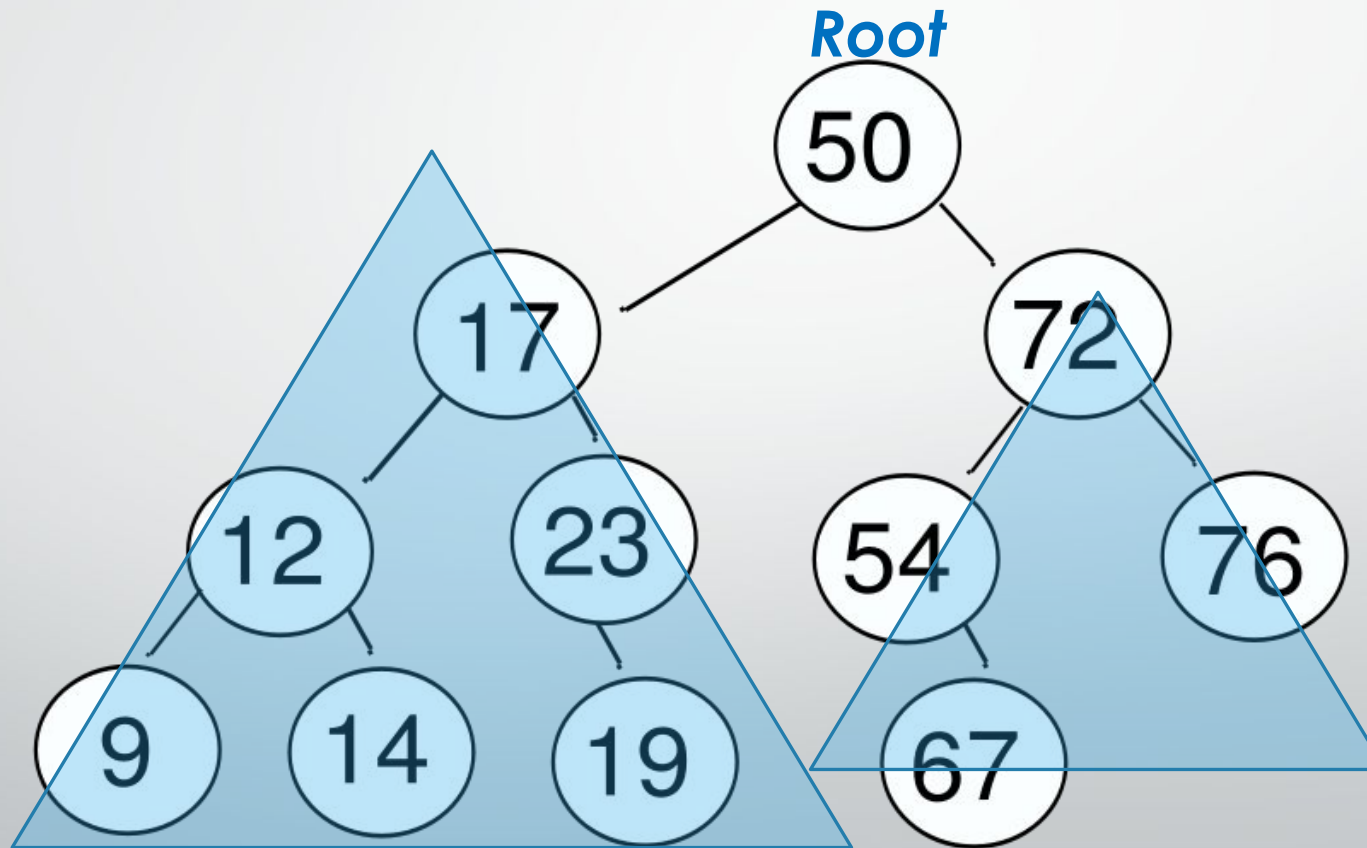
Trees: Properties

- ✓ The Tree is a recursive data structure:



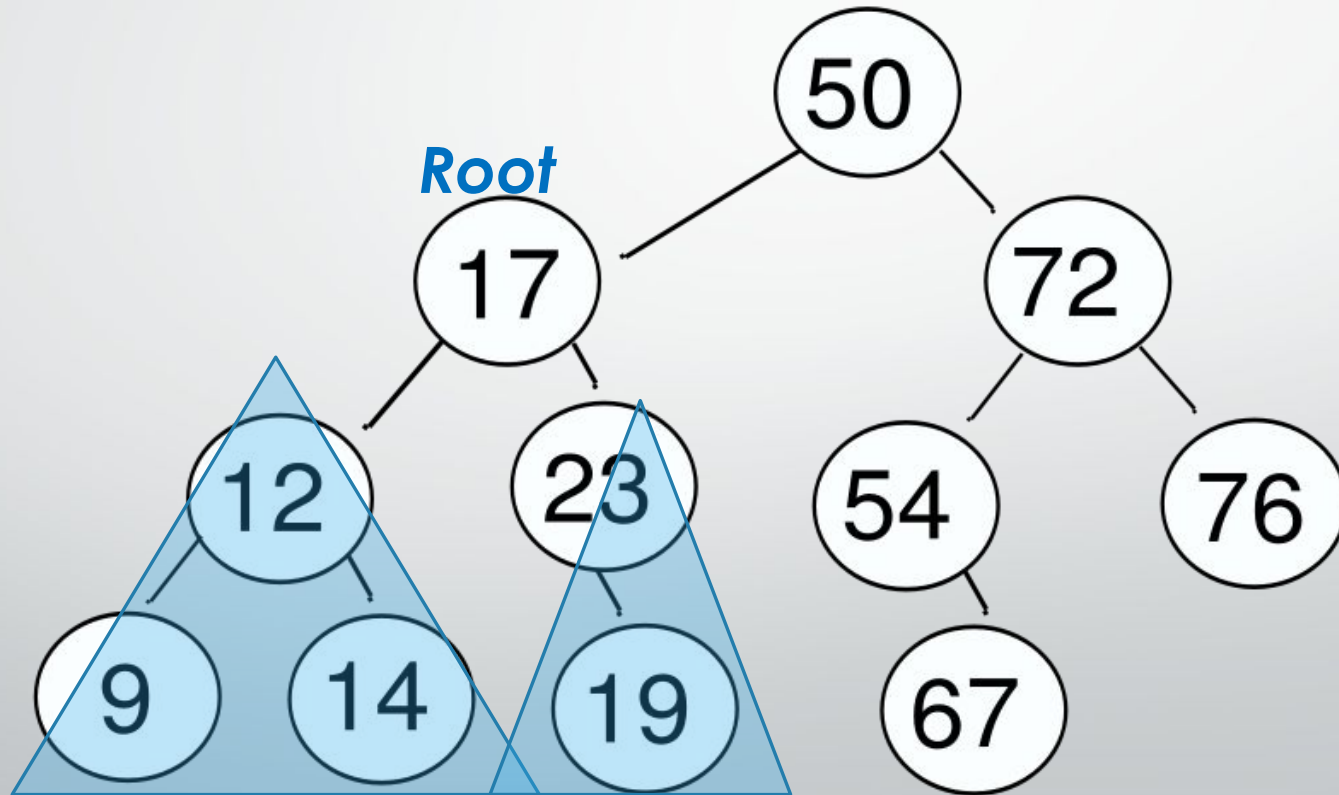
Trees: Properties

- ✓ The Tree is a recursive data structure:



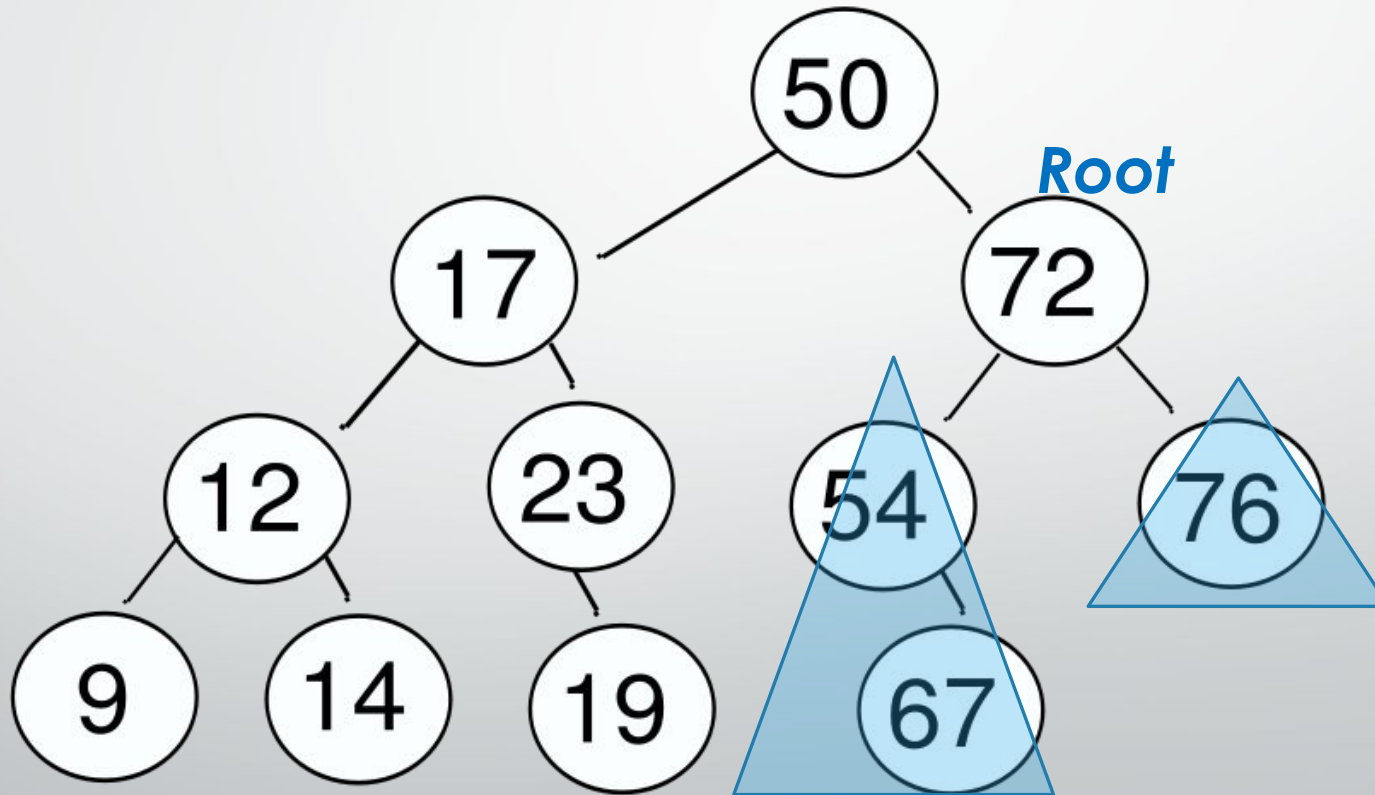
Trees: Properties

- ✓ The Tree is a recursive data structure:



Trees: Properties

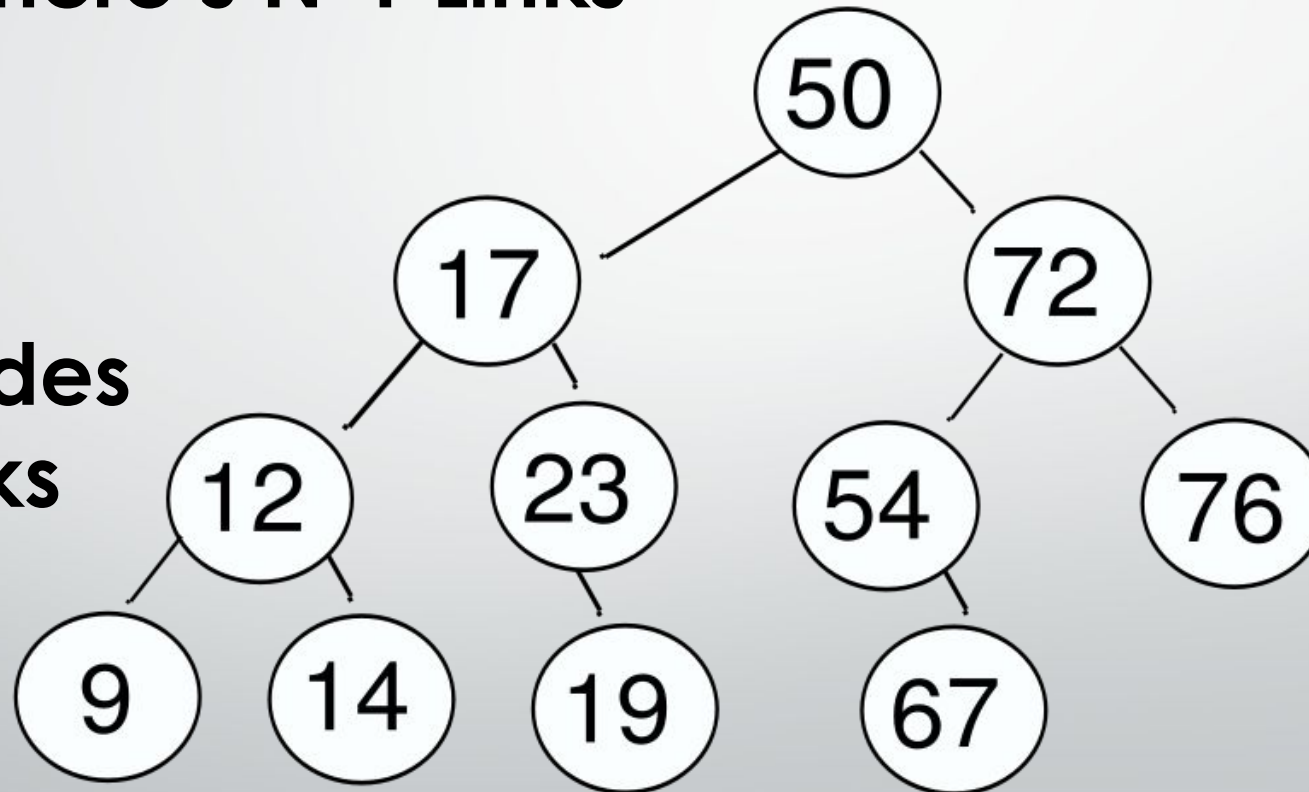
- ✓ The Tree is a recursive data structure:



Trees: Properties

- ✓ If the tree have N Nodes ,
- ✓ Then there's N-1 Links

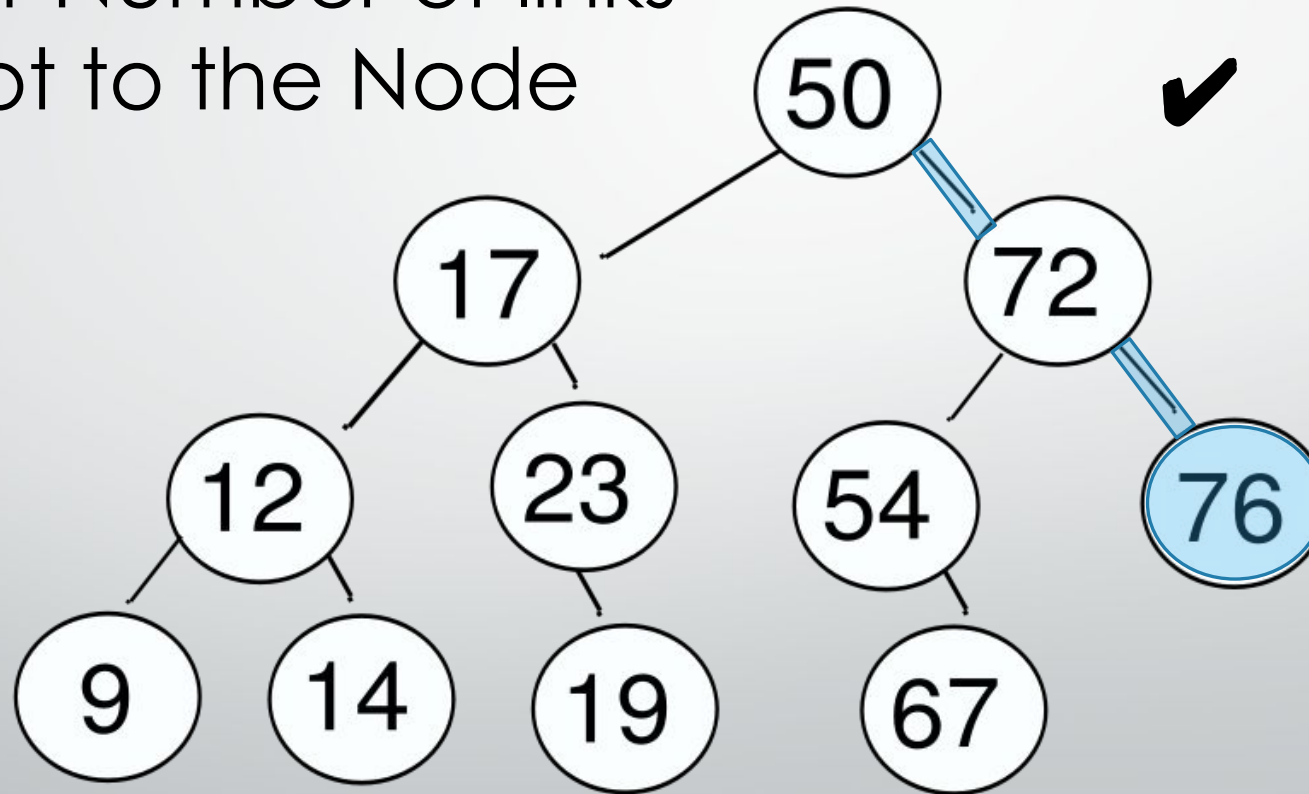
- ✓ 11 Nodes
- ✓ 10 Links



Trees: Properties

✓ Depth of a Node:

Length or Number of links
From Root to the Node

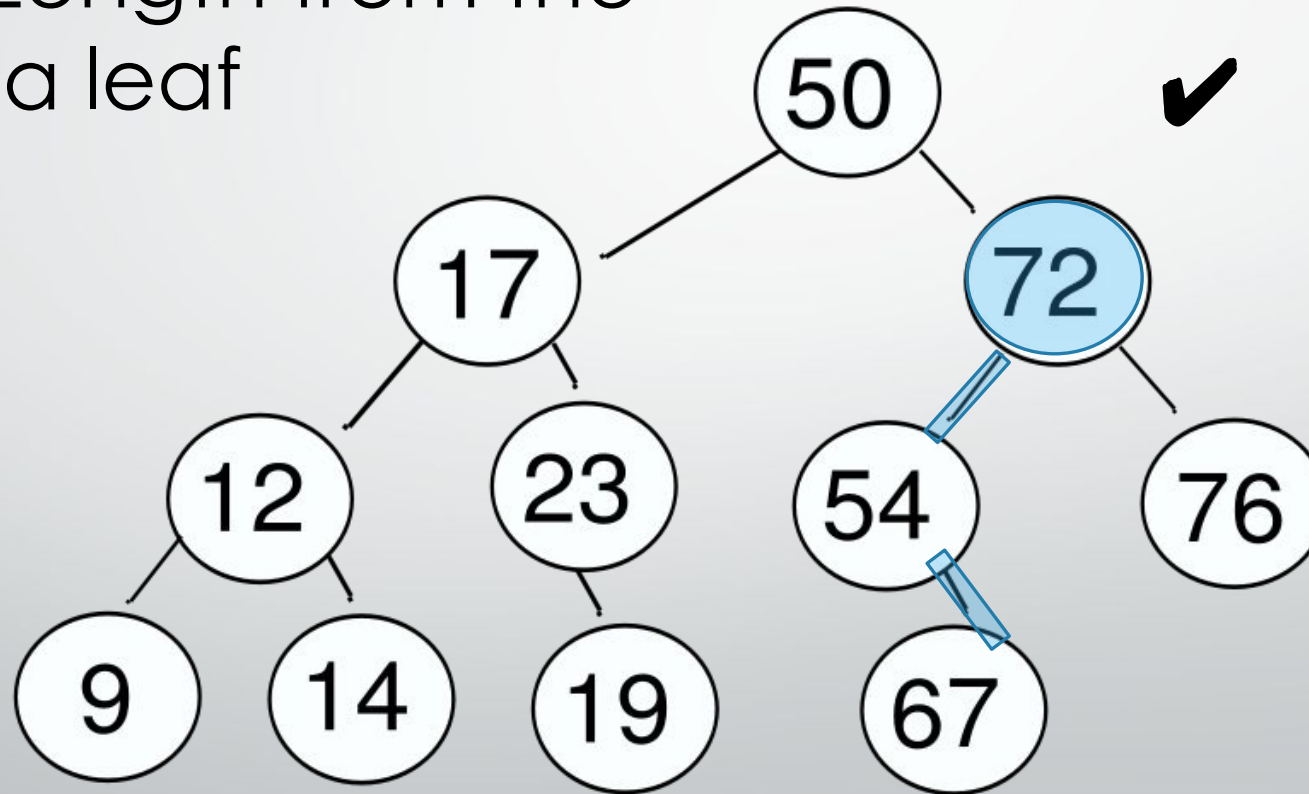


✓ Ex:
Depth of
Node 76 = 2

Trees: Properties

✓ Height of a Node:

Longest Length from the Node to a leaf

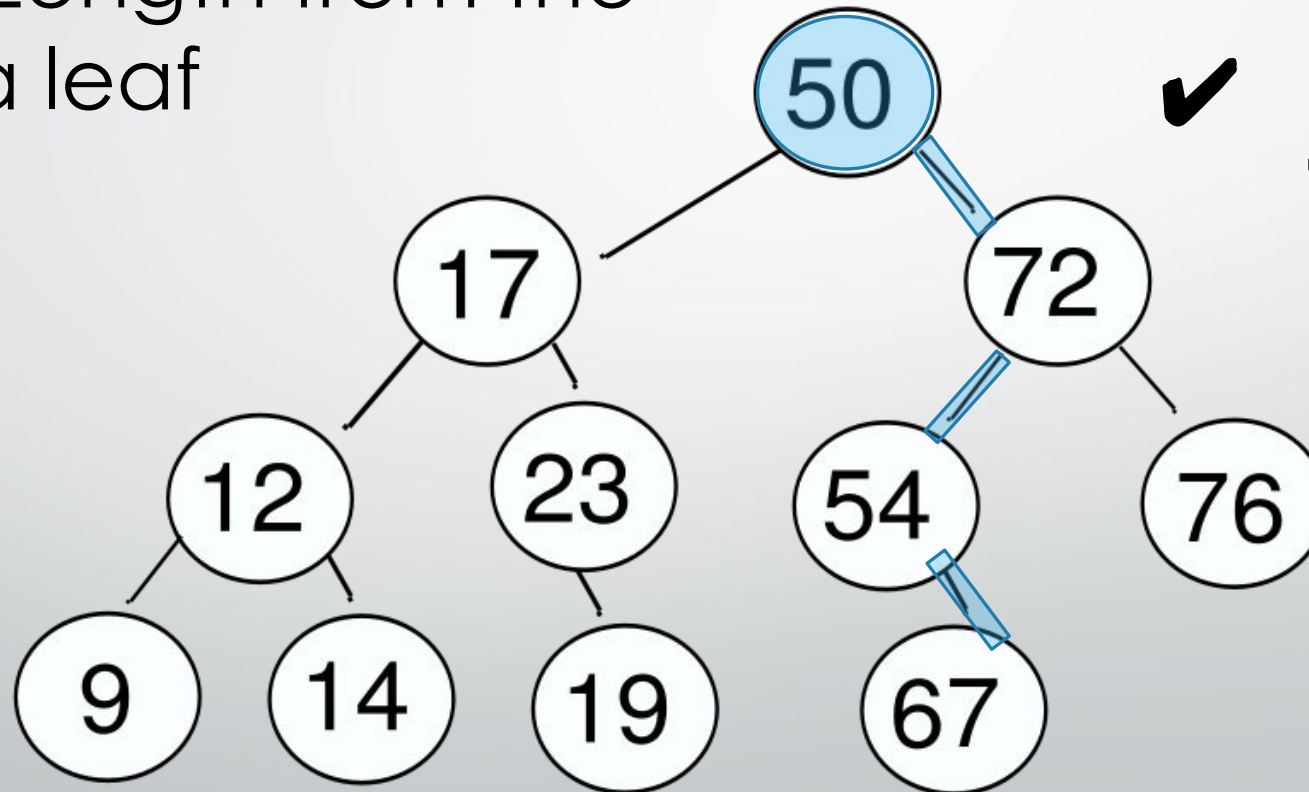


✓ Ex:
Height of
Node 72 = 2

Trees: Properties

✓ Height of a Tree:

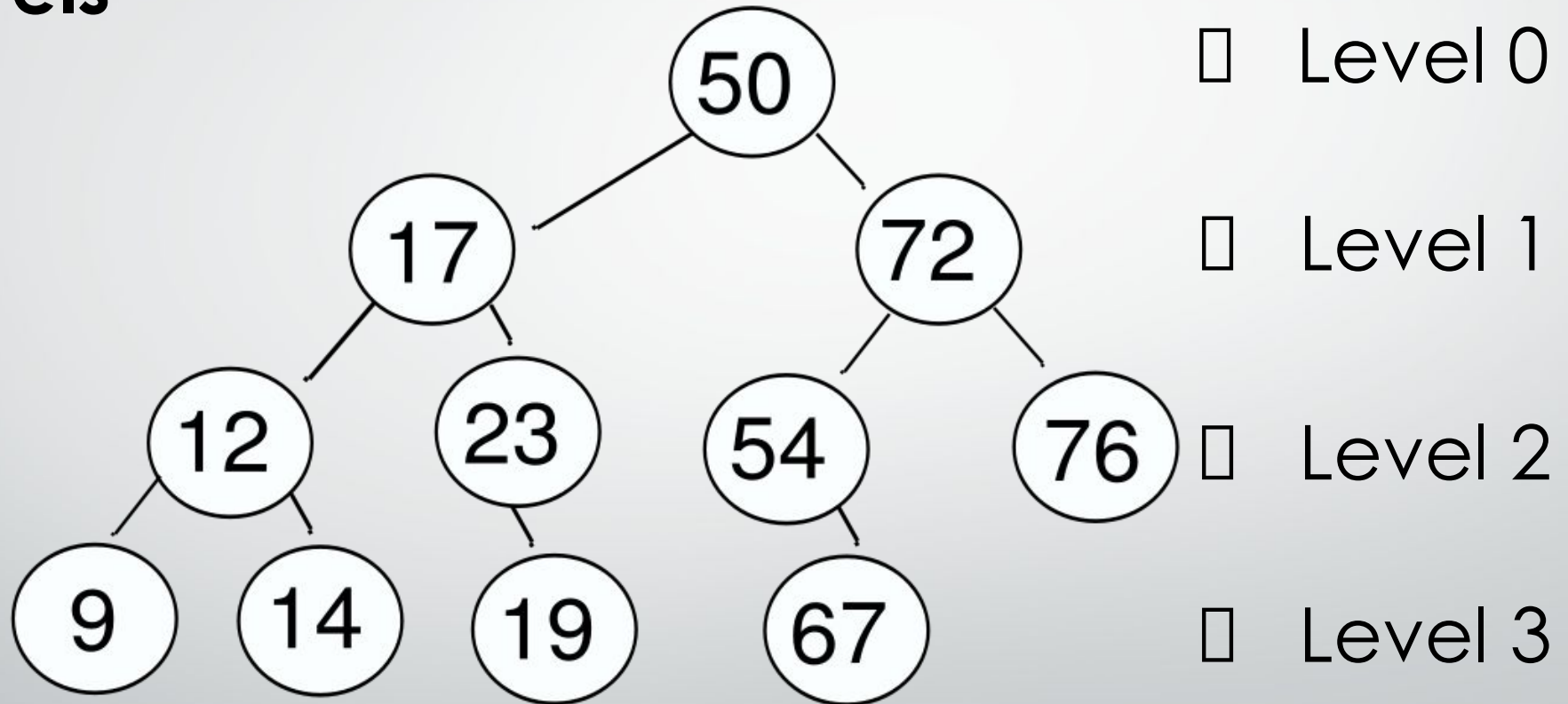
Longest Length from the Root to a leaf



✓ Height of this Tree = 3

Trees: Properties

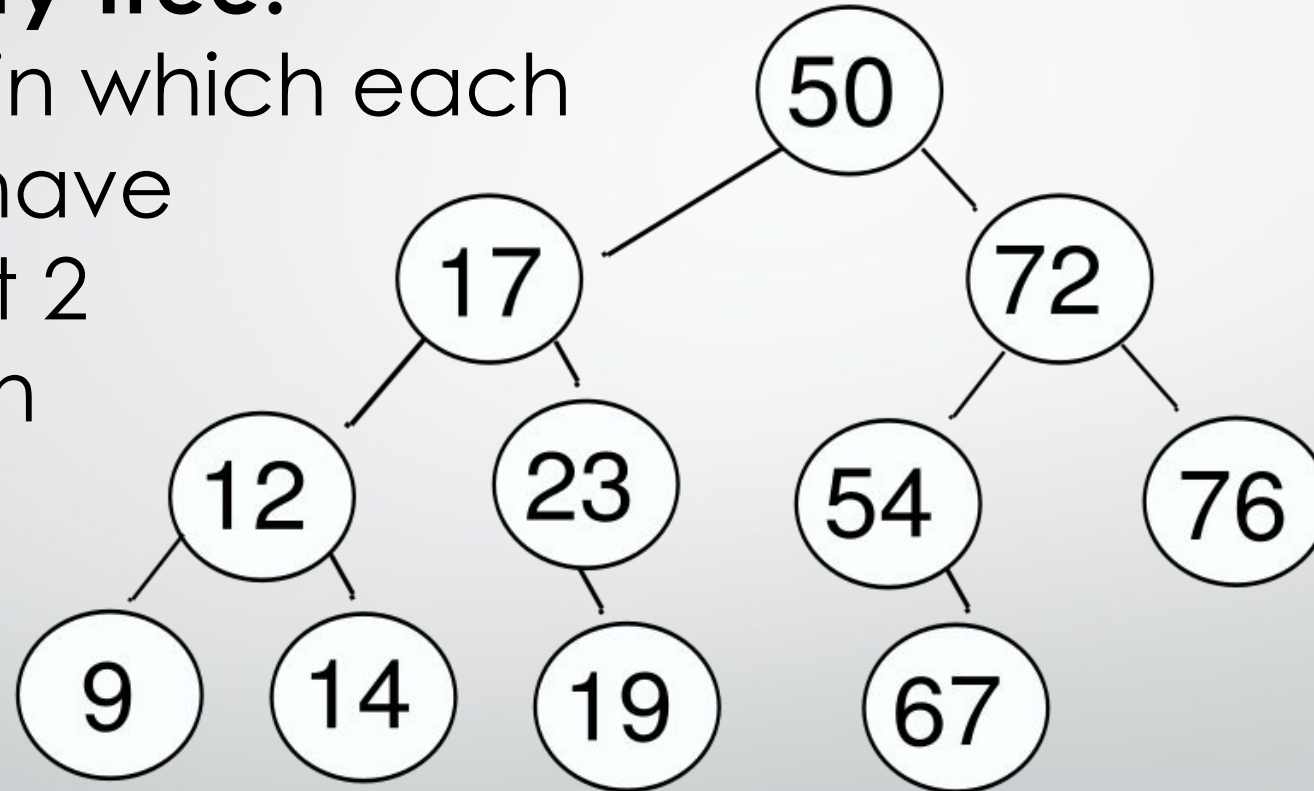
✓ Levels



Binary Tree

✓ Binary Tree:

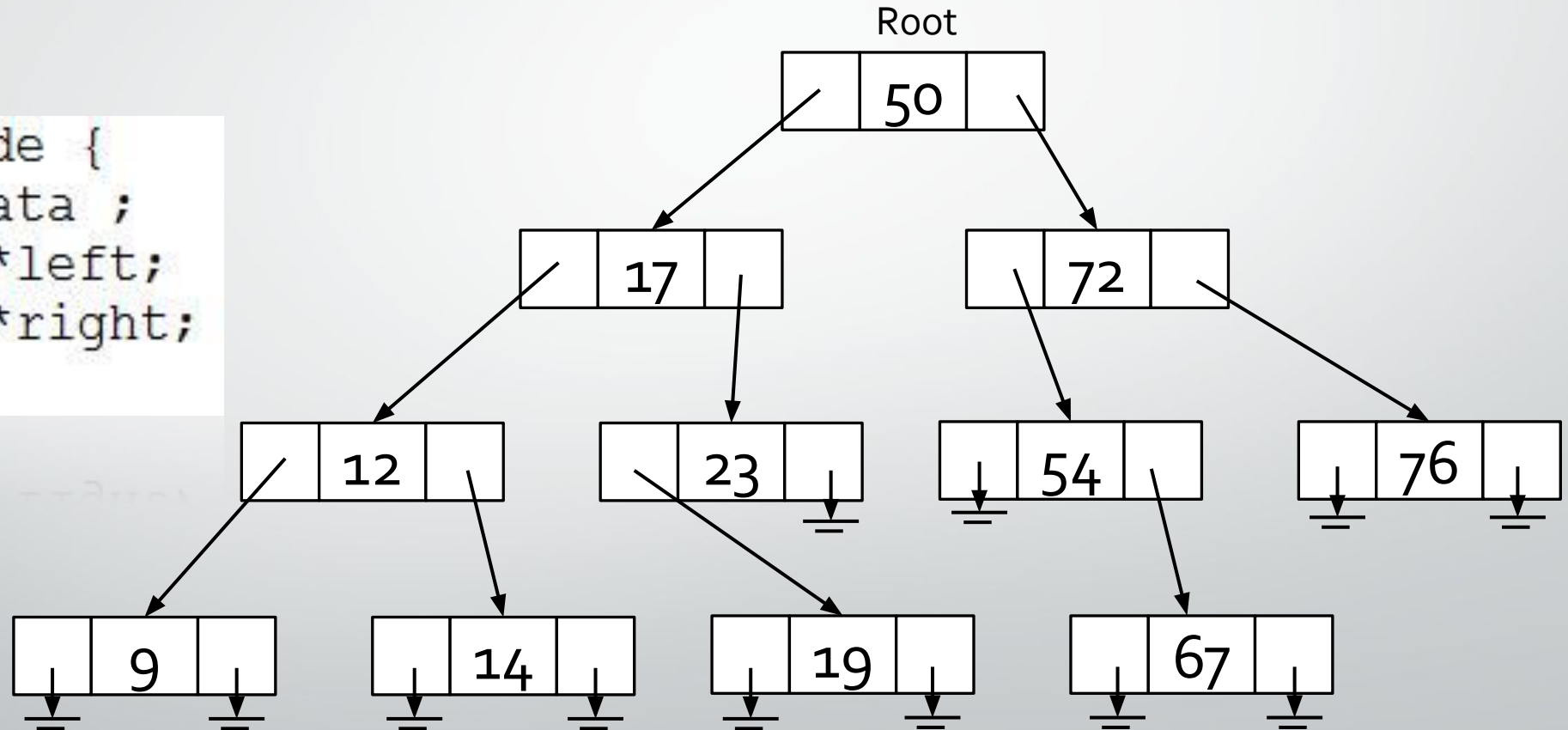
A Tree in which each Node have At most 2 children



Binary Tree

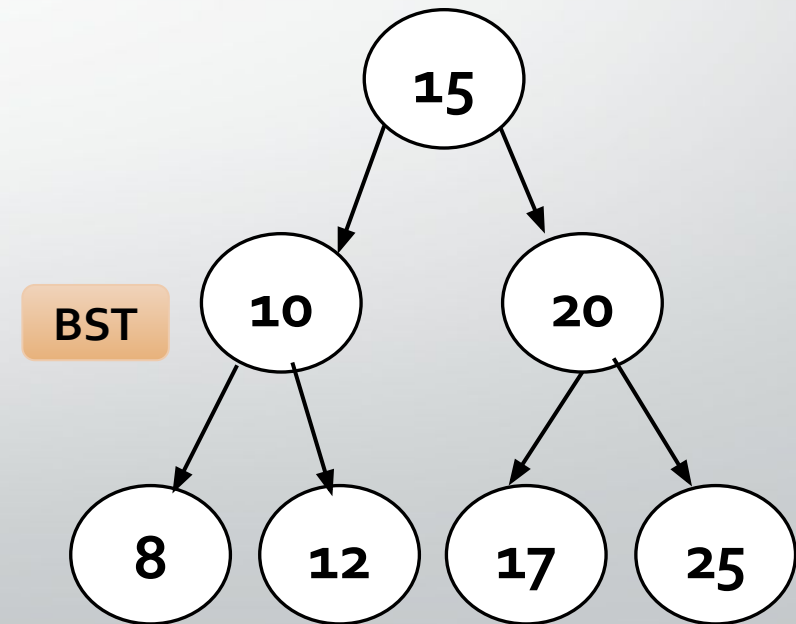
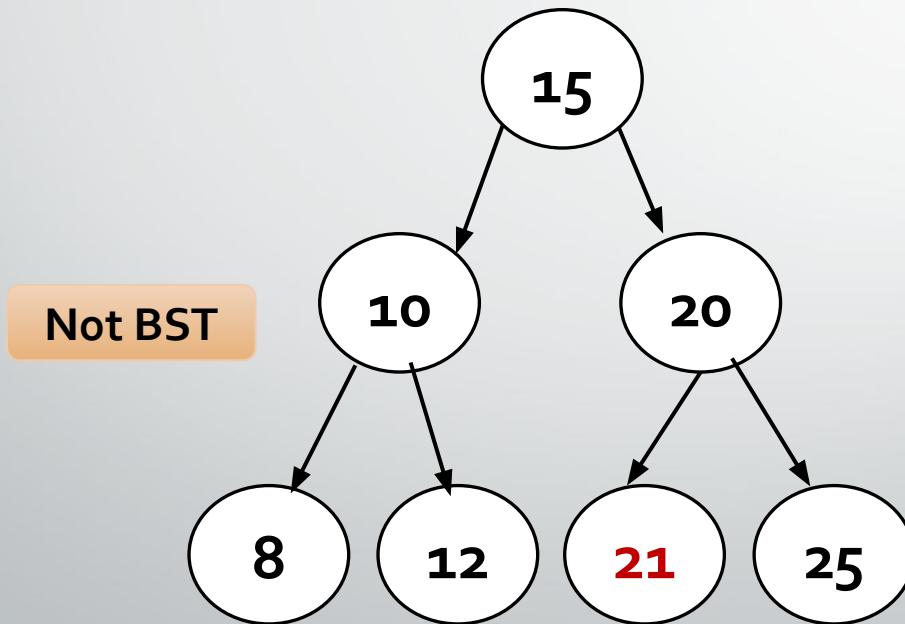
✓ If we use linked list to represent it:

```
struct Node {  
    int data ;  
    Node *left;  
    Node *right;  
};
```



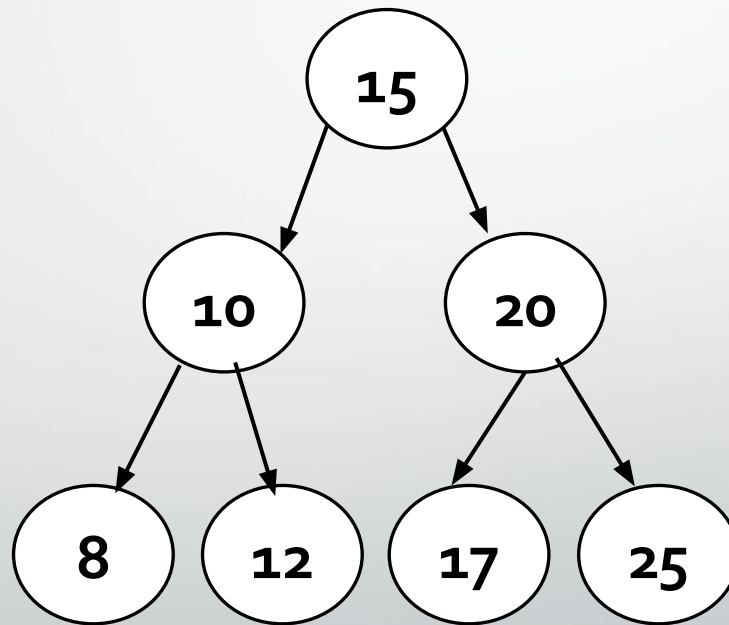
Binary Search Tree

- ✓ for each node the value of all nodes in left sub-tree is lesser or equal , and values in right sub-tree is greater

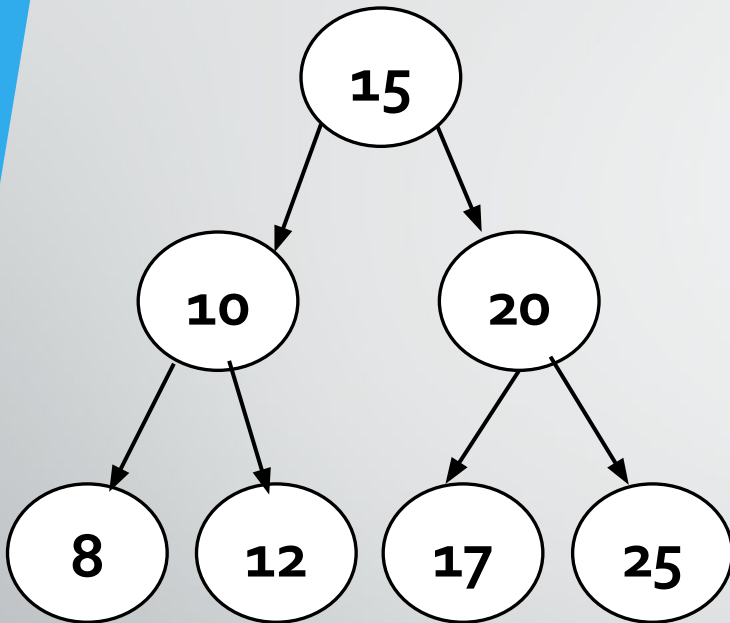


Insert Node in a BST

✓ How we can Insert a Node in this BST ?



Insert Node in a BST

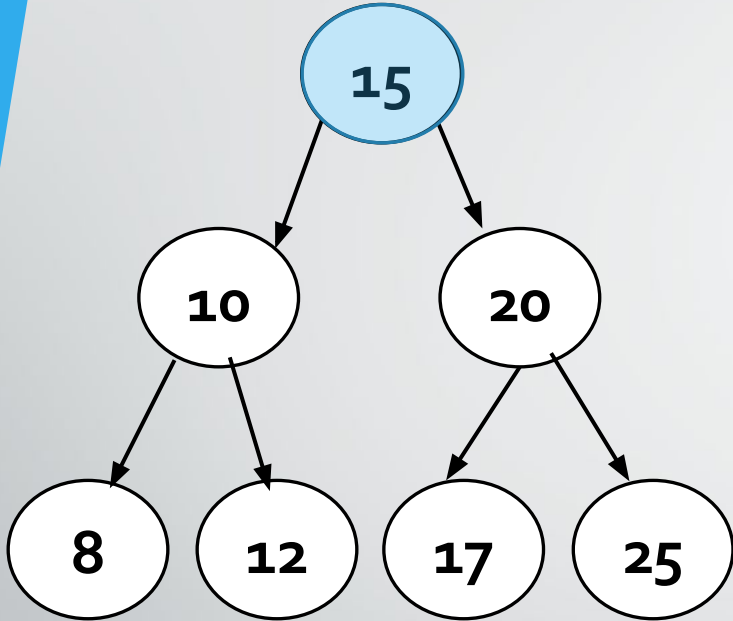


```
void insert(Node *&root, int x) {  
    if (root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if (x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```



Ex: Insert 19

Insert Node in a BST



```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

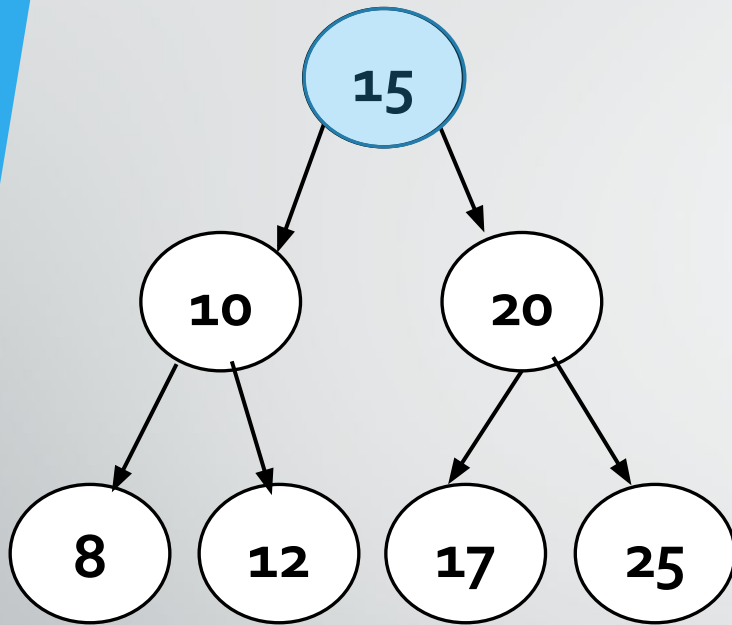
NO



Ex: Insert 19

Insert Node in a BST

9



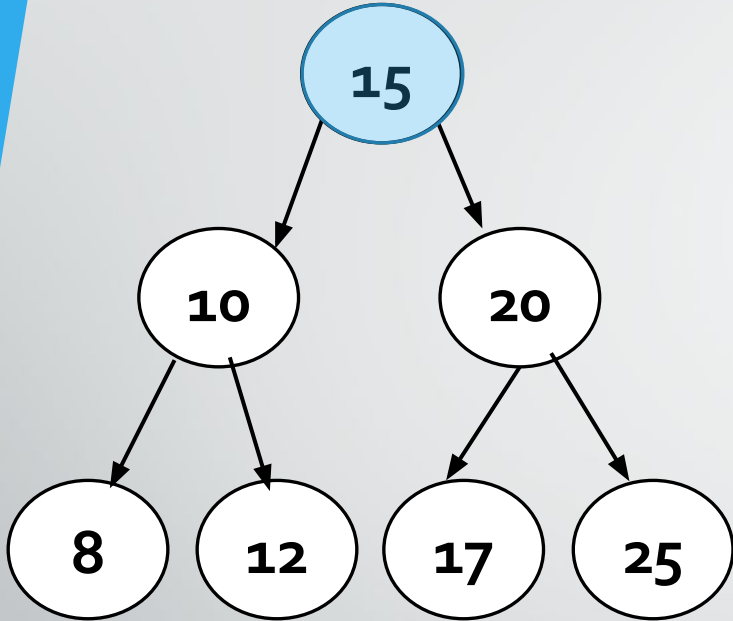
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

NO



Ex: Insert 19

Insert Node in a BST



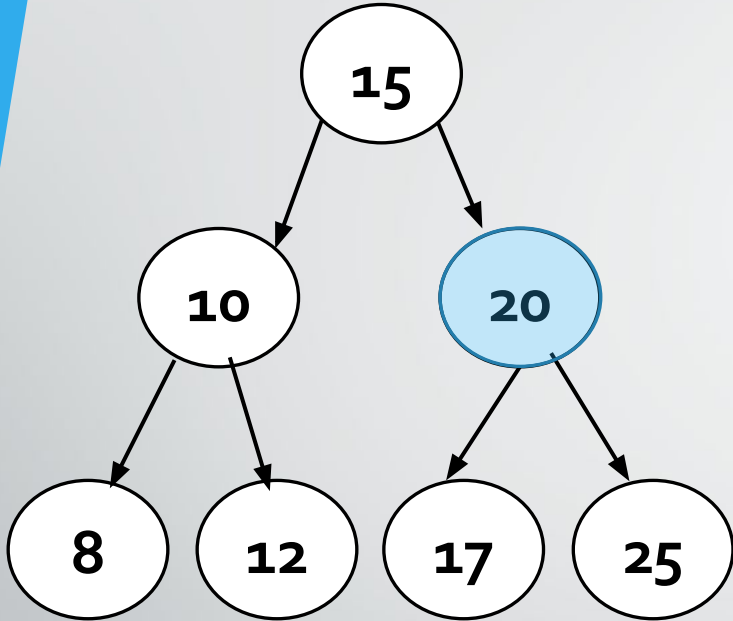
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES



Ex: Insert 19

Insert Node in a BST



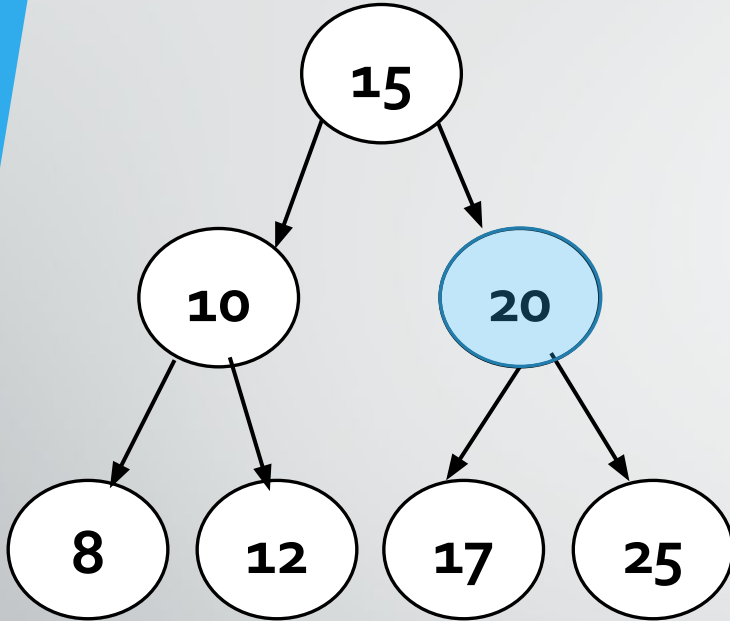
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES



Ex: Insert 19

Insert Node in a BST



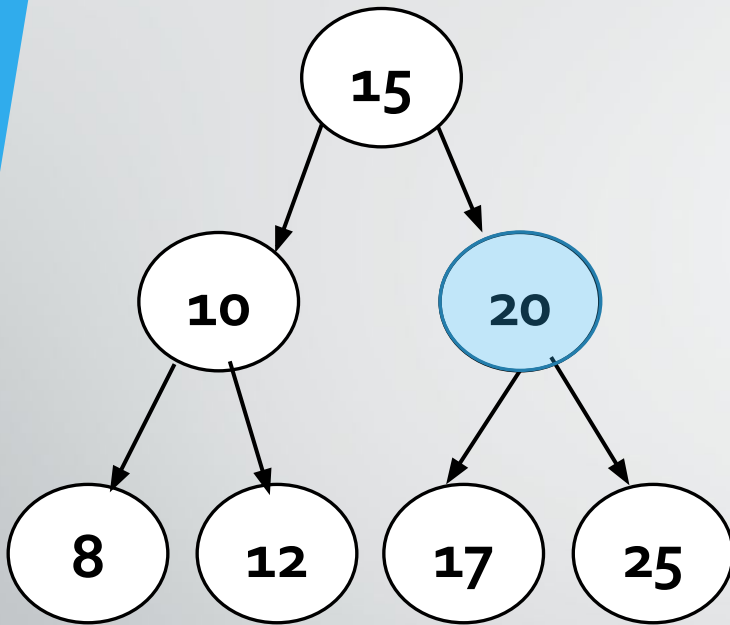
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

NO



Ex: Insert 19

Insert Node in a BST



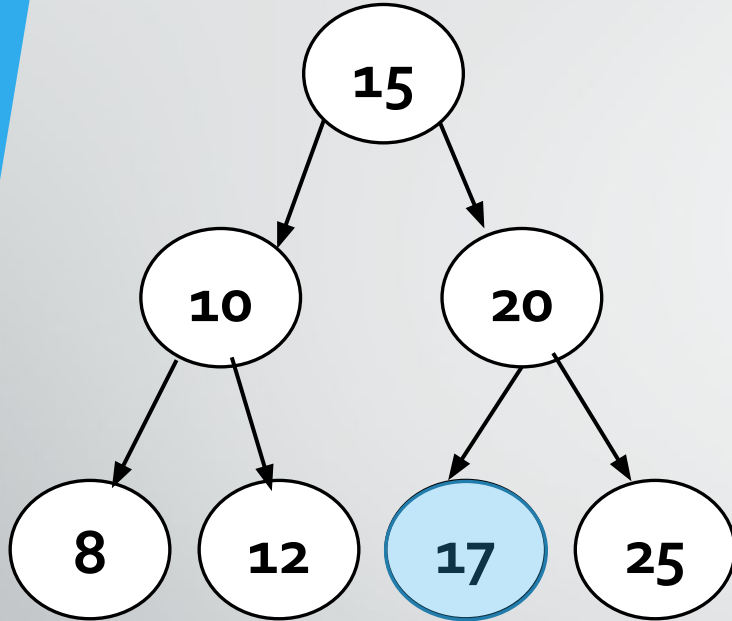
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES



Ex: Insert 19

Insert Node in a BST



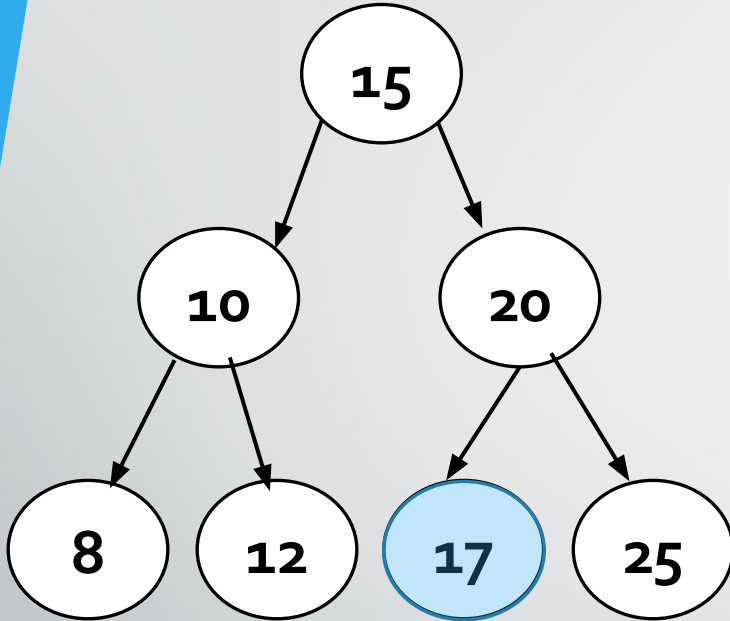
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES



Ex: Insert 19

Insert Node in a BST



```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

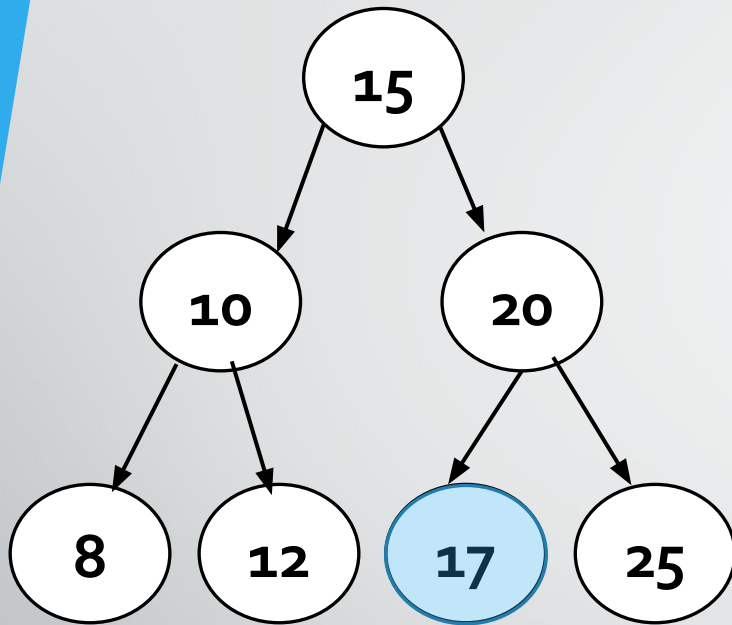
NO



Ex: Insert 19

Insert Node in a BST

9



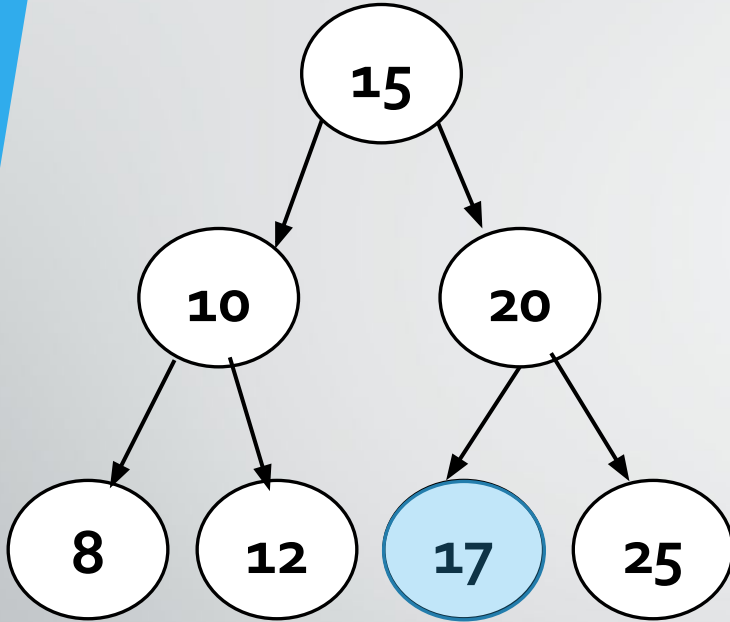
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

NO



Ex: Insert 19

Insert Node in a BST



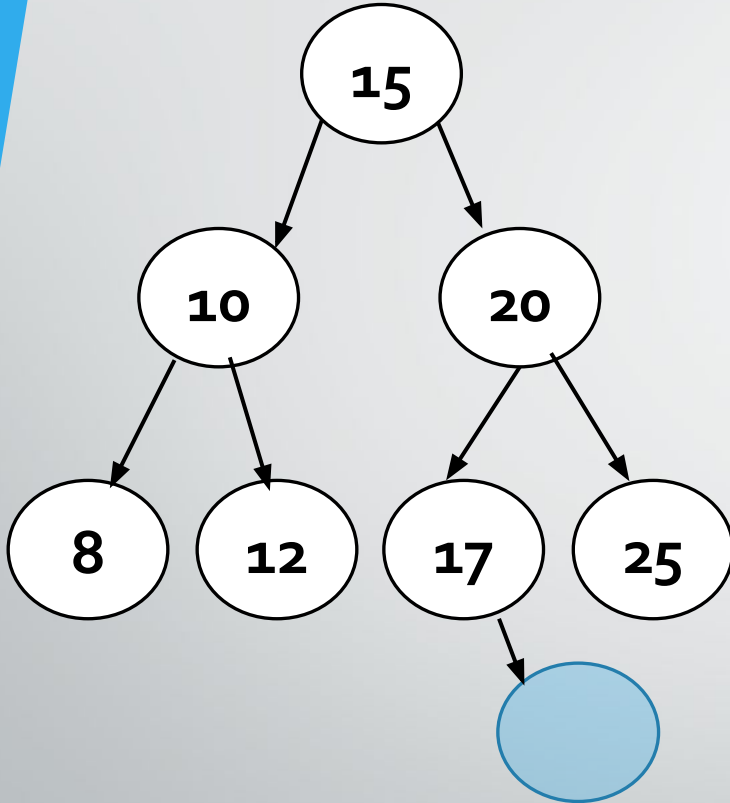
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES



Ex: Insert 19

Insert Node in a BST



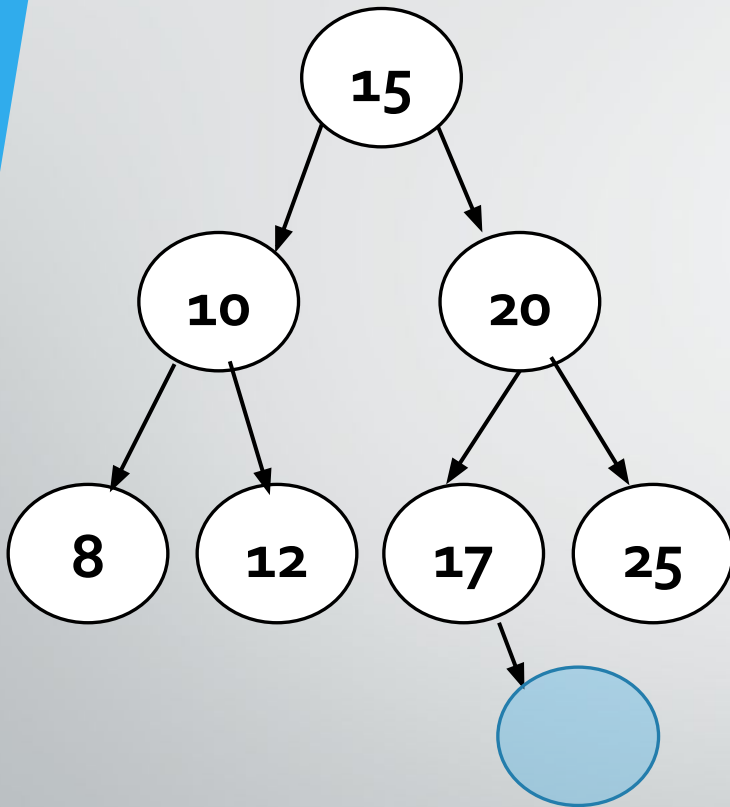
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES



Ex: Insert 19

Insert Node in a BST



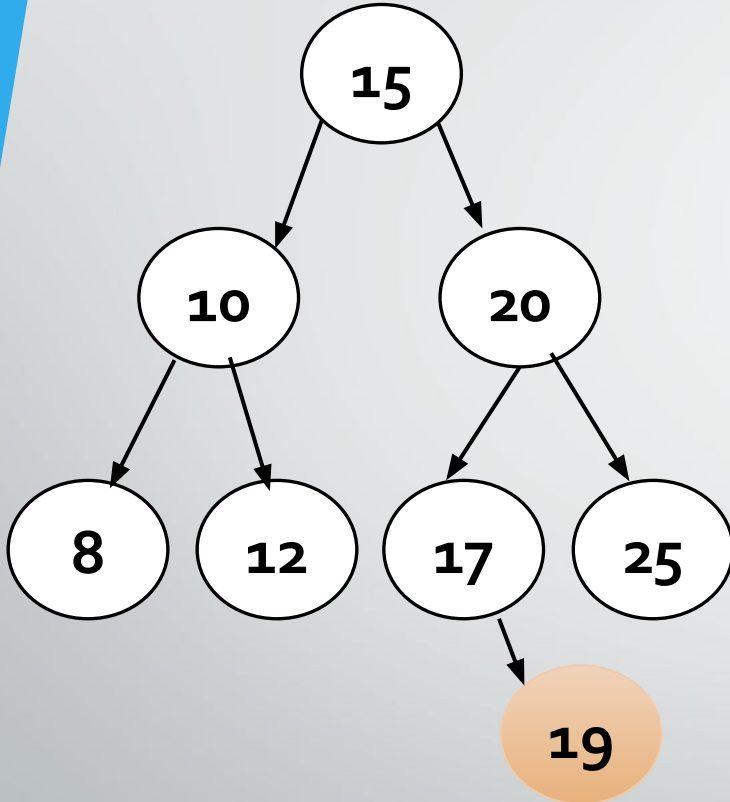
```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES



Ex: Insert 19

Insert Node in a BST

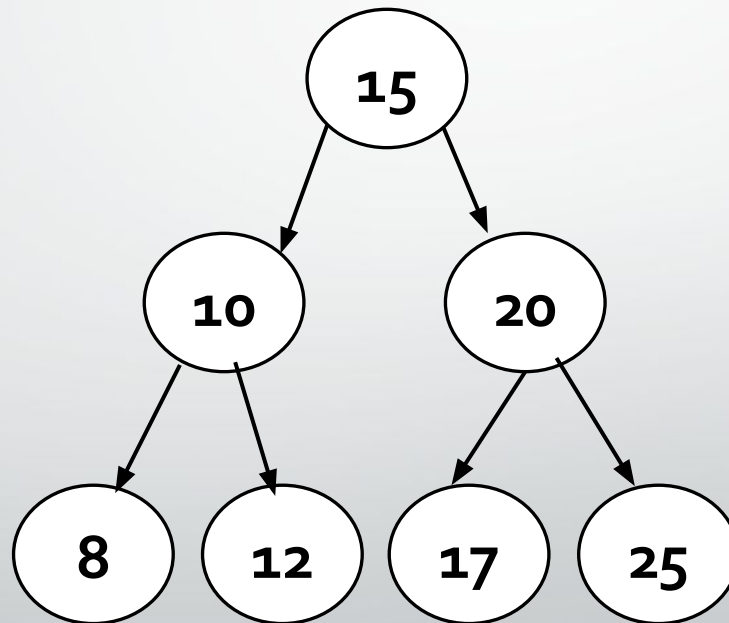


```
void insert(Node *&root, int x) {  
    if(root == NULL) {  
        Node *temp = new Node();  
        temp->data = x;  
        temp->left = NULL;  
        temp->right = NULL;  
        root = temp;  
    }  
    else if(x <= root->data) {  
        insert(root->left, x);  
    }  
    else {  
        insert(root->right, x);  
    }  
}
```

YES

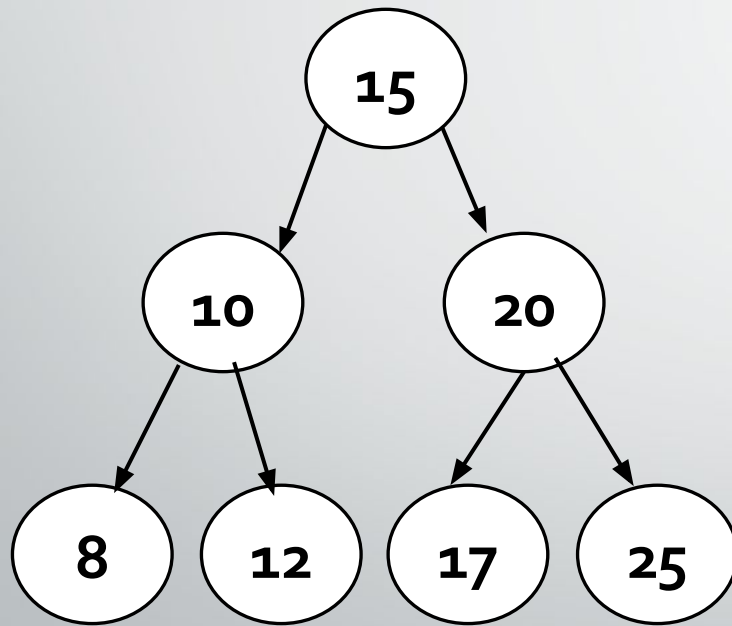
Search for a number in BST

- ✓ **How we can search for a number in a BST ?**
- ✓ The solution is similar to what we did in insertion , (using recursion)



Search for a number in BST

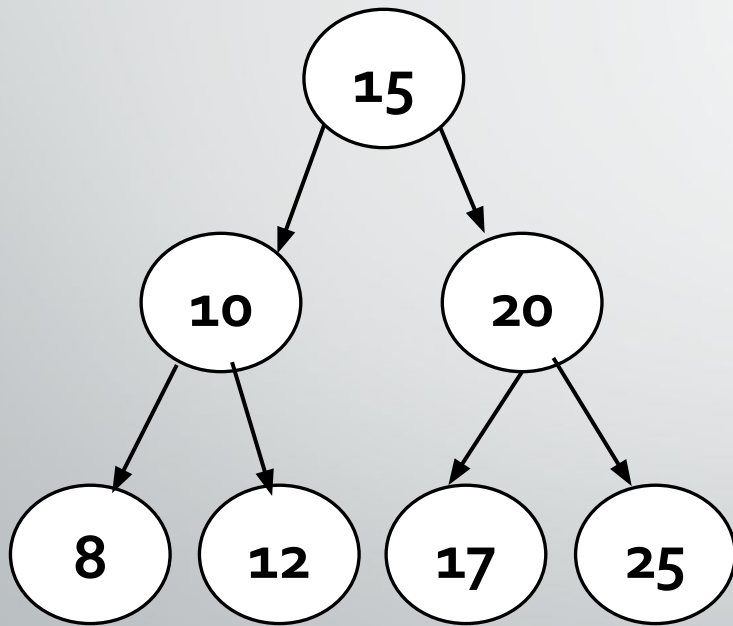
- ✓ First , Check if the root is empty or not , then print “not found”



```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
}
```

Search for a number in BST

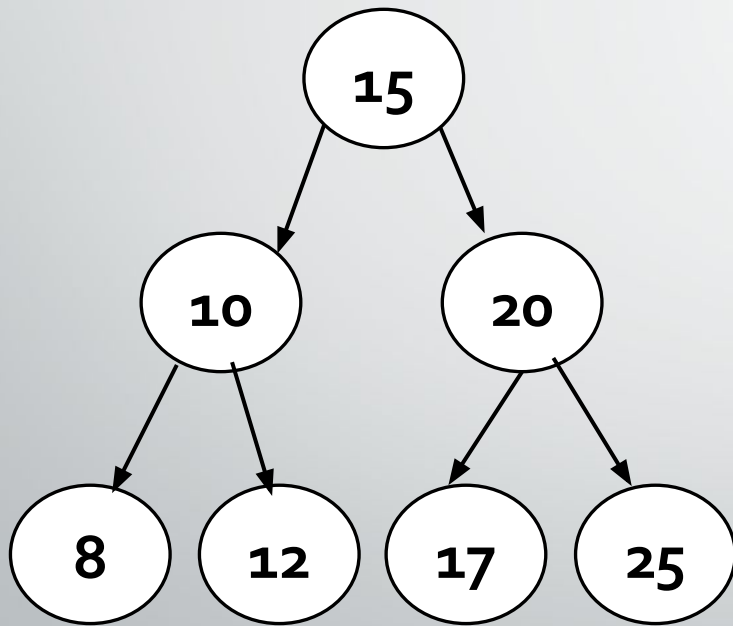
- ✓ If the target is equal to the root , then it's found



```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
}
```


Search for a number in BST

- ✓ Check if the target is $<$ the root , then search in the left sub-tree , else search right

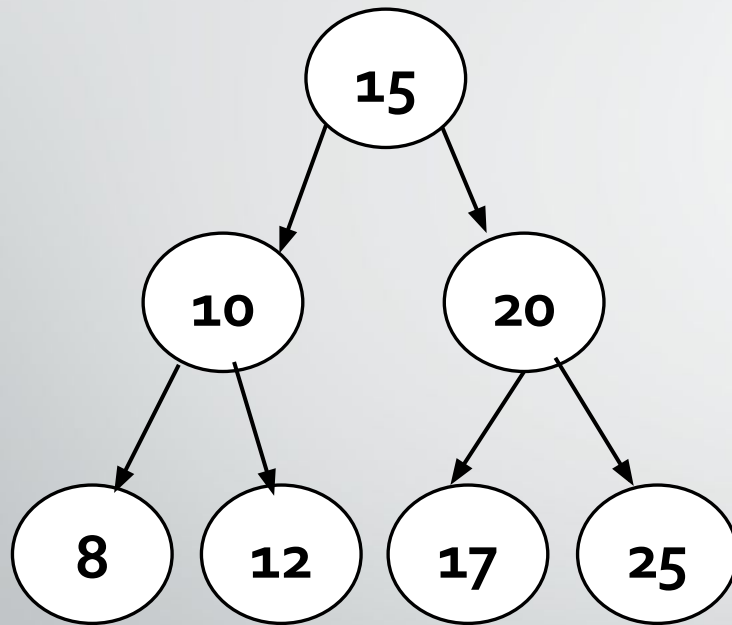


```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

Search for a number in BST



Ex: Search for 12

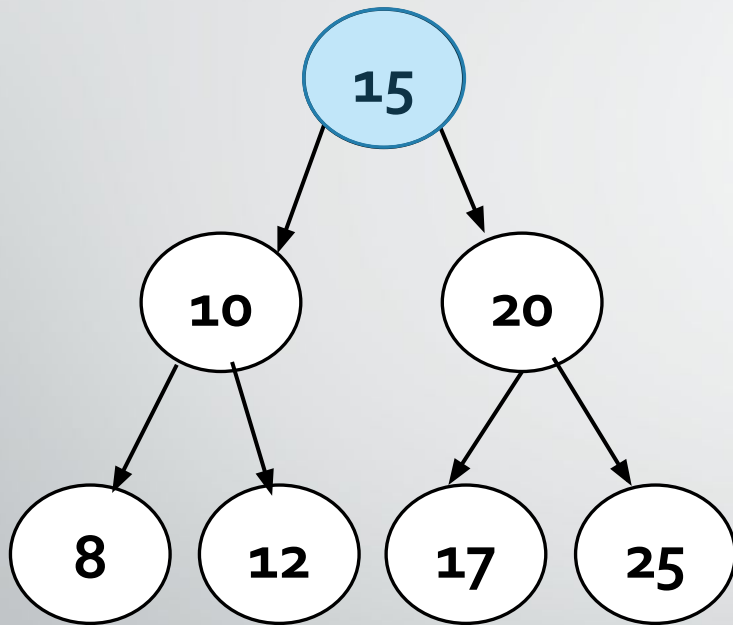


```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

Search for a number in BST



Ex: Search for 12



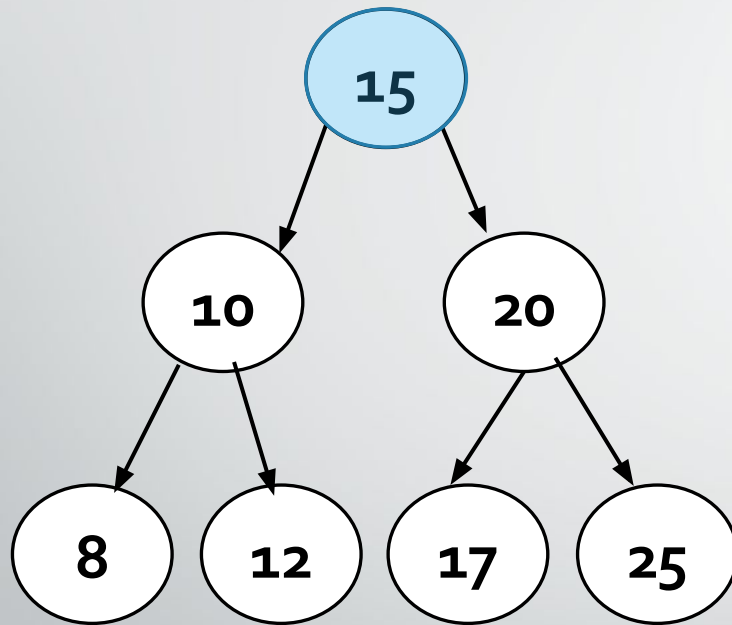
```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

NO

Search for a number in BST



Ex: Search for 12

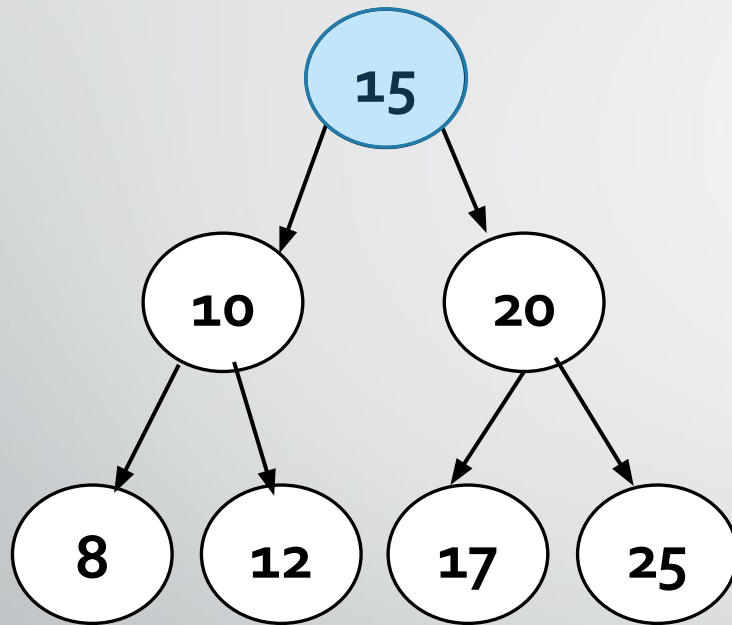


```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

NO

Search for a number in BST

✓ Ex: Search for 12



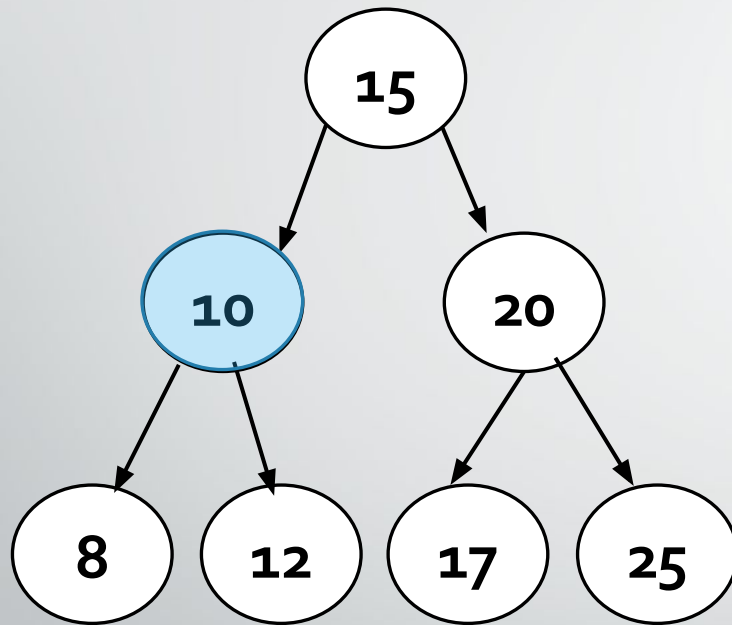
```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

YES

Search for a number in BST



Ex: Search for 12



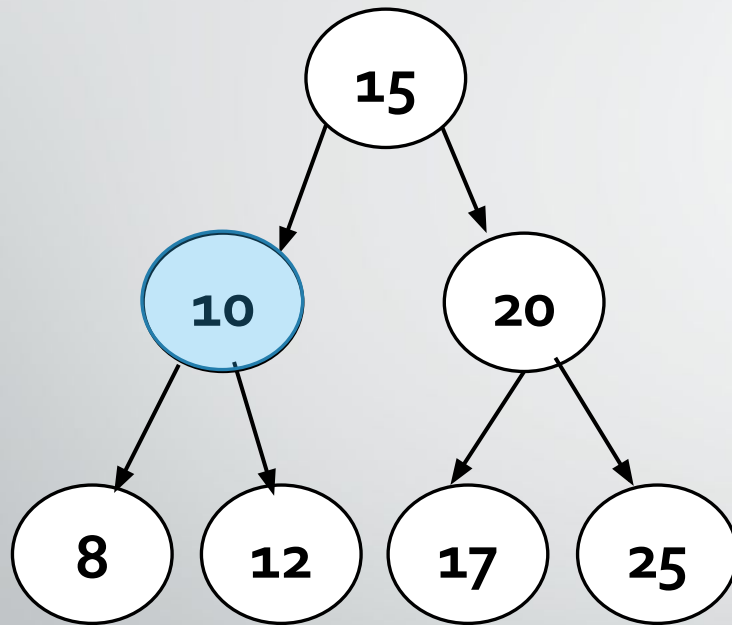
```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

YES

Search for a number in BST



Ex: Search for 12



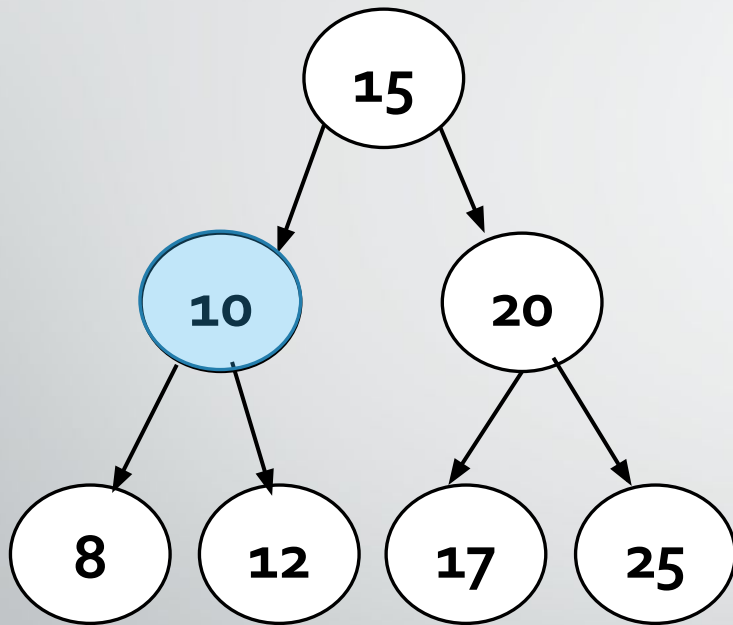
```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

NO

Search for a number in BST



Ex: Search for 12



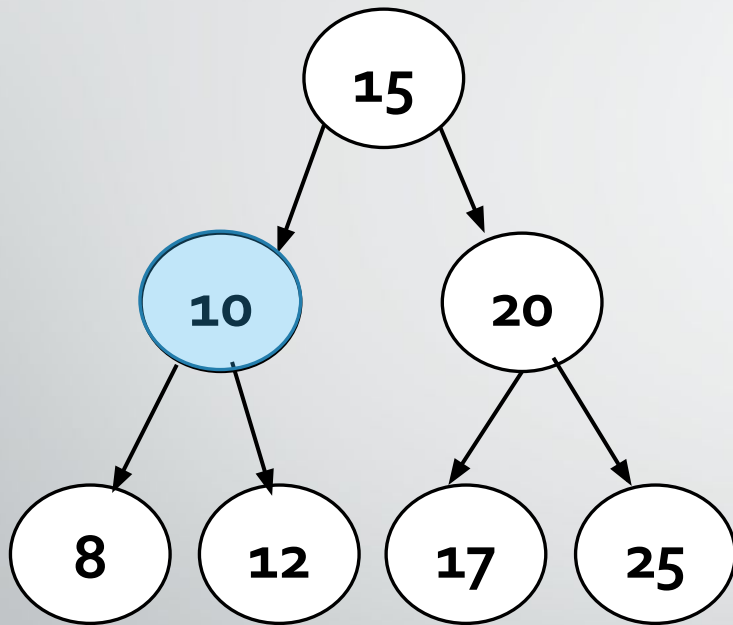
```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

NO

Search for a number in BST



Ex: Search for 12



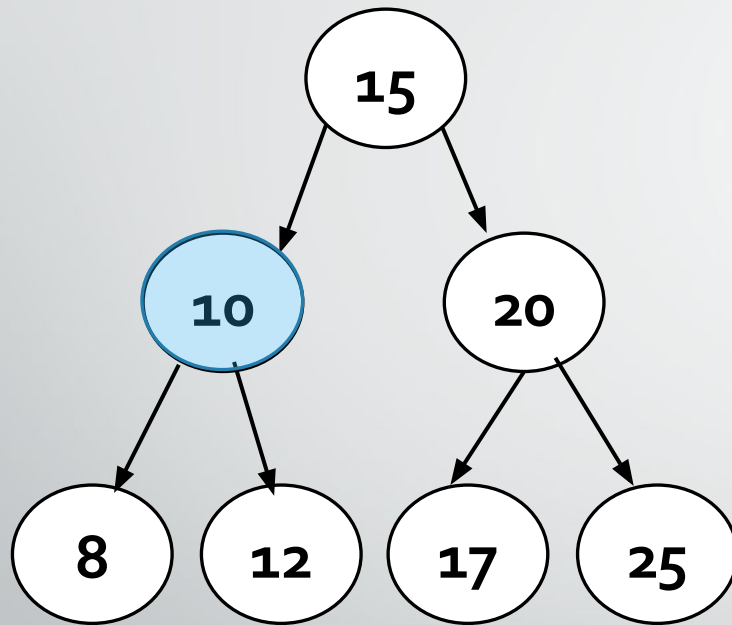
```
void search(Node *&root, int x){
    if(root == NULL){
        printf("Not Found");
        return;
    }
    else if(x == root->data){
        printf("Found");
        return;
    }
    else if(x < root->data){
        search(root->left, x);
    }
    else{
        search(root->right, x);
    }
}
```

NO

Search for a number in BST



Ex: Search for 12

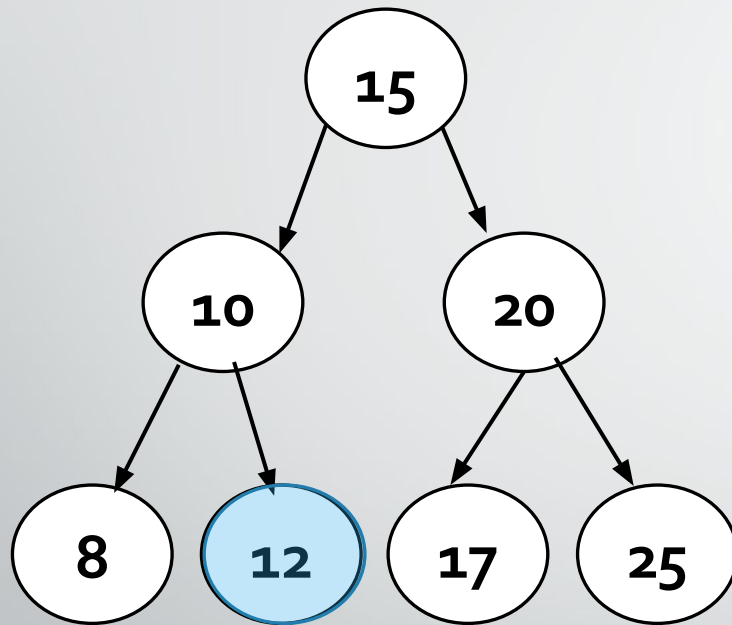


```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

YES

Search for a number in BST

✓ Ex: Search for 12



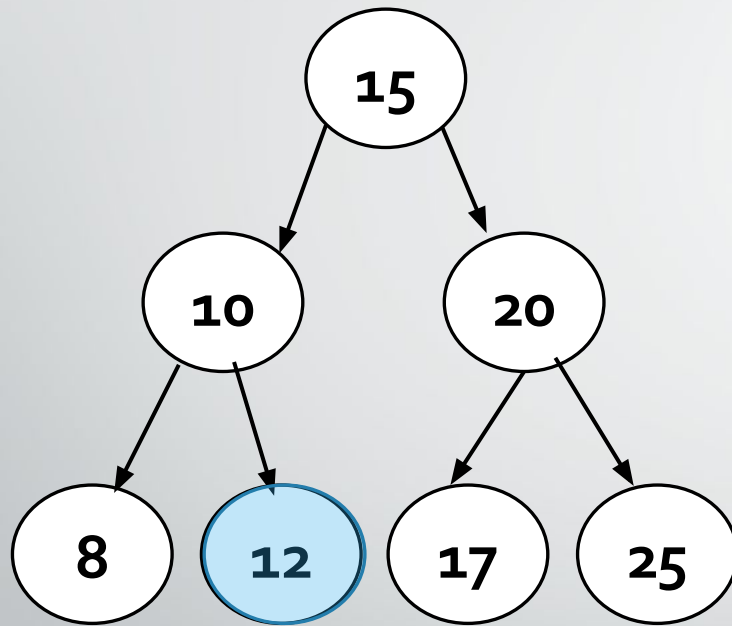
```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

YES

Search for a number in BST



Ex: Search for 12



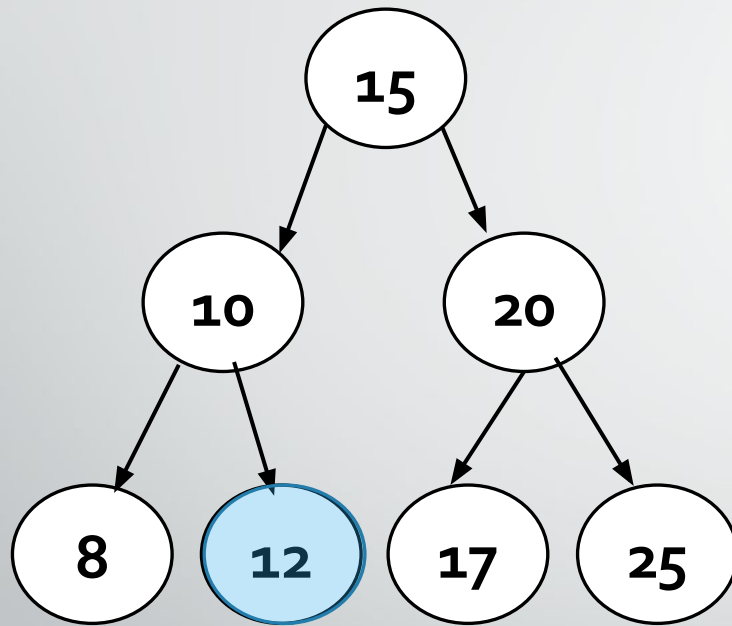
```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

NO

Search for a number in BST



Ex: Search for 12

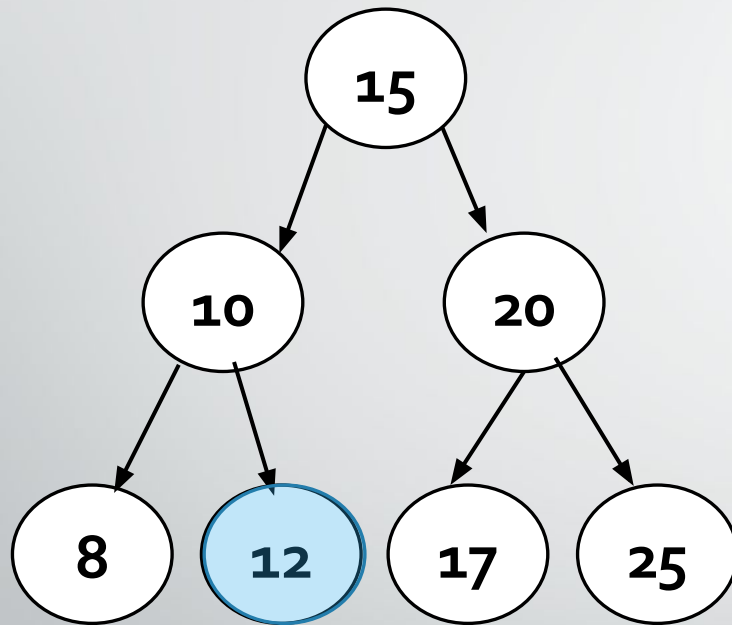


```
void search(Node *&root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

YES

Search for a number in BST

✓ **Ex: Search for 12**



✓ **Found**

```
void search(Node *root, int x){  
    if(root == NULL){  
        printf("Not Found");  
        return;  
    }  
    else if(x == root->data){  
        printf("Found");  
        return;  
    }  
    else if(x < root->data){  
        search(root->left, x);  
    }  
    else{  
        search(root->right, x);  
    }  
}
```

YES

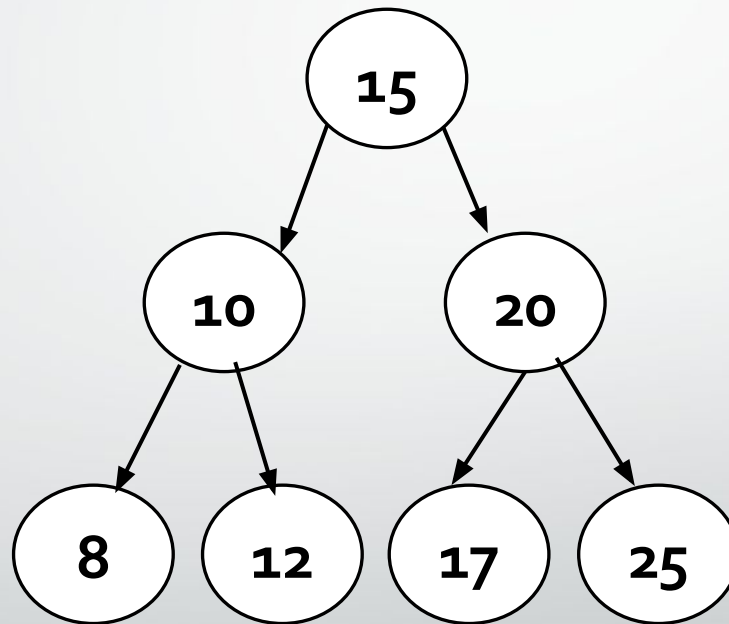
Search for a number in BST

✓ **Another Way**

```
Node* search(Node *&root, int x) {  
    if(root == NULL) {  
        return root;  
    }  
    else if(x == root->data) {  
        return root;  
    }  
    else if(x < root->data) {  
        return search(root->left, x);  
    }  
    else {  
        return search(root->right, x);  
    }  
}
```

Count elements in the BST

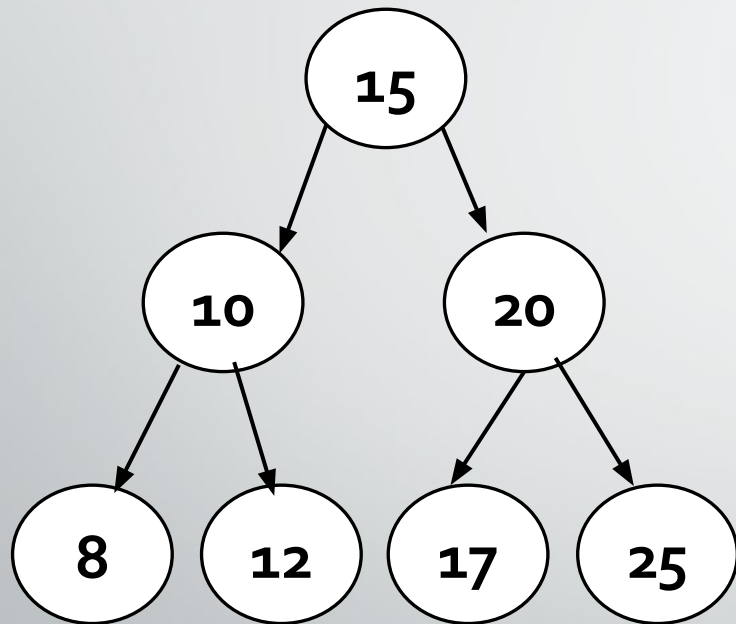
✓ How we can count nodes in a BST ?



Count elements in the BST

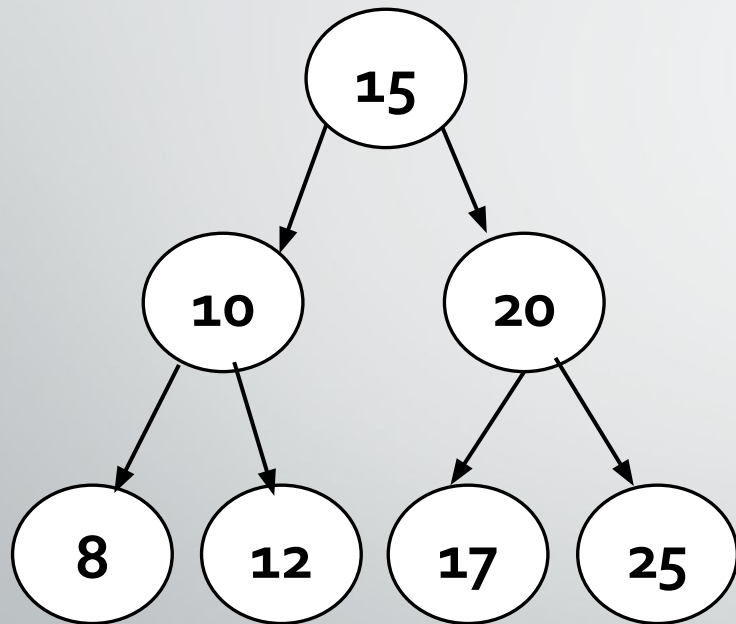
- ✓ First , Check if root is empty then return 0

```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
}
```



Count elements in the BST

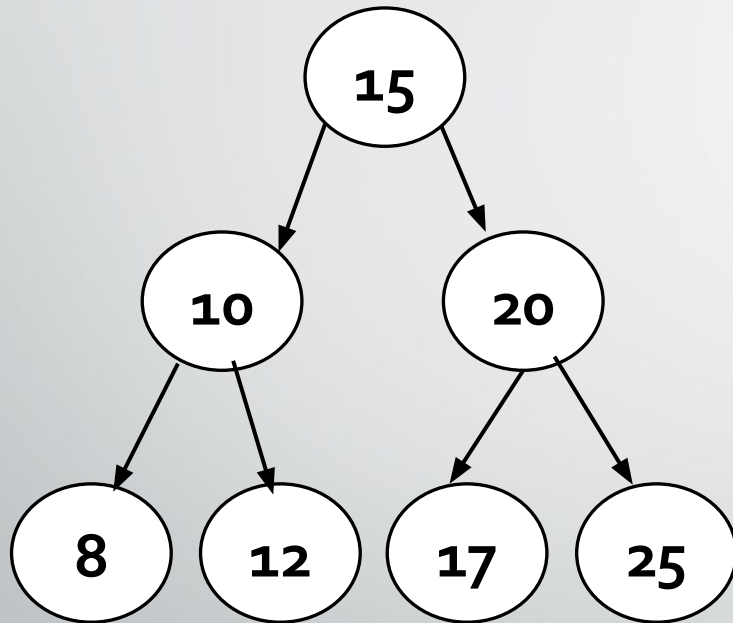
- ✓ Else , return 1 + count the left Subtree + count the right



```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

Count elements in the BST

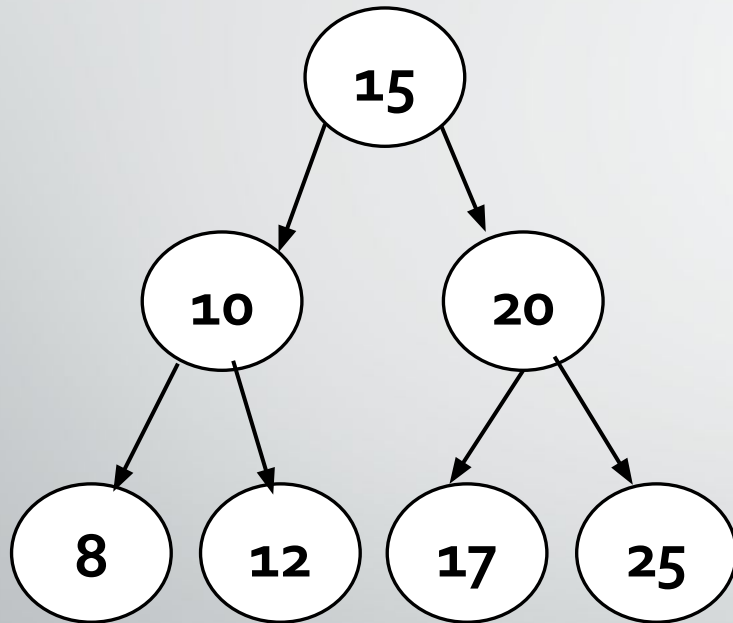
✓ Ex: Count this tree



```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

Count elements in the BST

✓ Ex: Count this tree

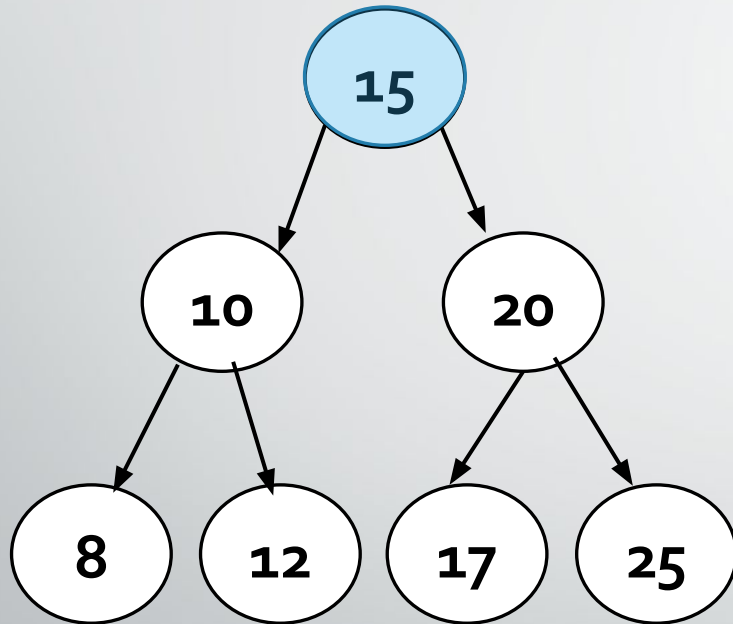


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)

Count elements in the BST

✓ Ex: Count this tree

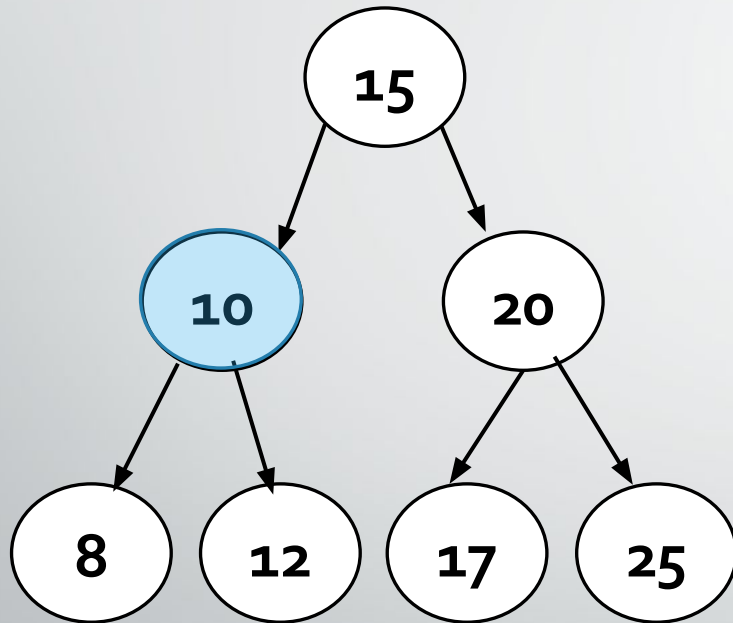


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)

Count elements in the BST

✓ Ex: Count this tree

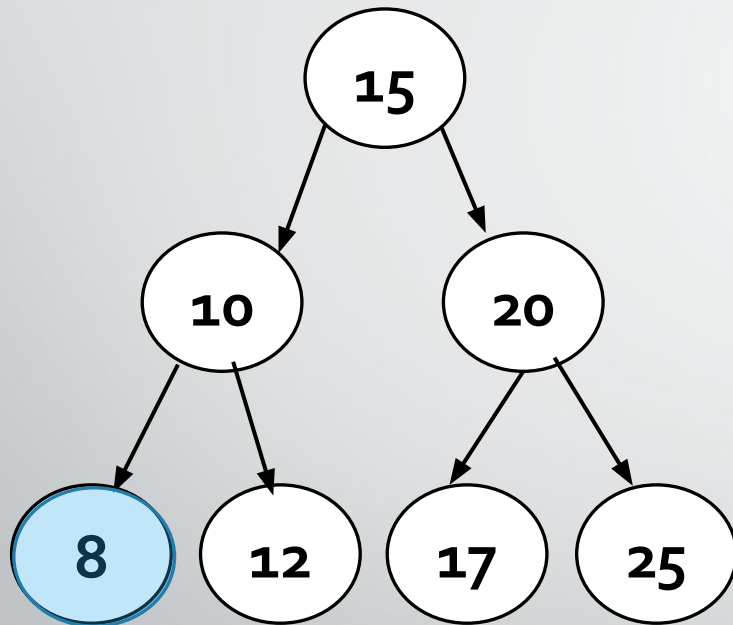


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+ 1+**count(8)+count(12)** +count(20)

Count elements in the BST

✓ Ex: Count this tree

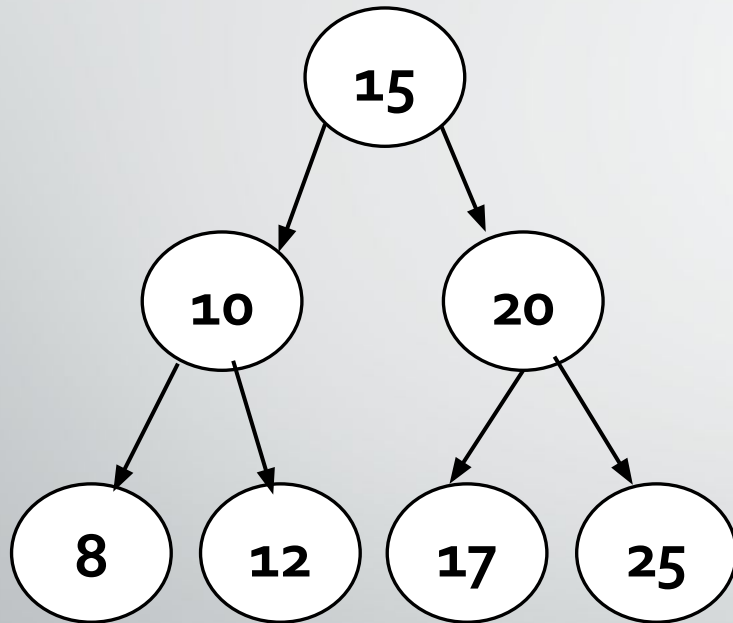


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+ **1+c(null)+c(null)** +c(12)+c(20)

Count elements in the BST

✓ Ex: Count this tree

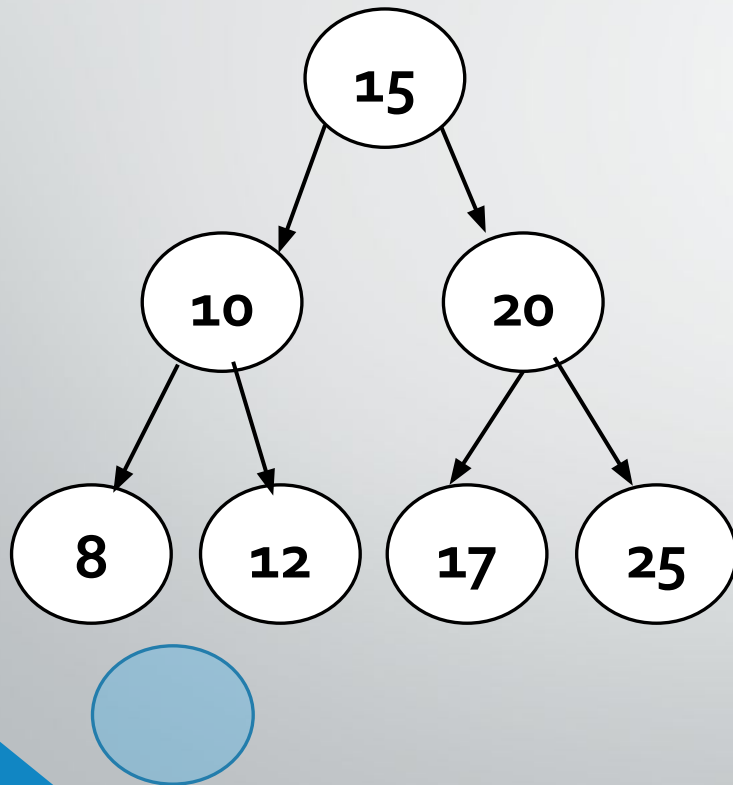


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+**0**+c(null)+c(12)+c(20)

Count elements in the BST

✓ Ex: Count this tree

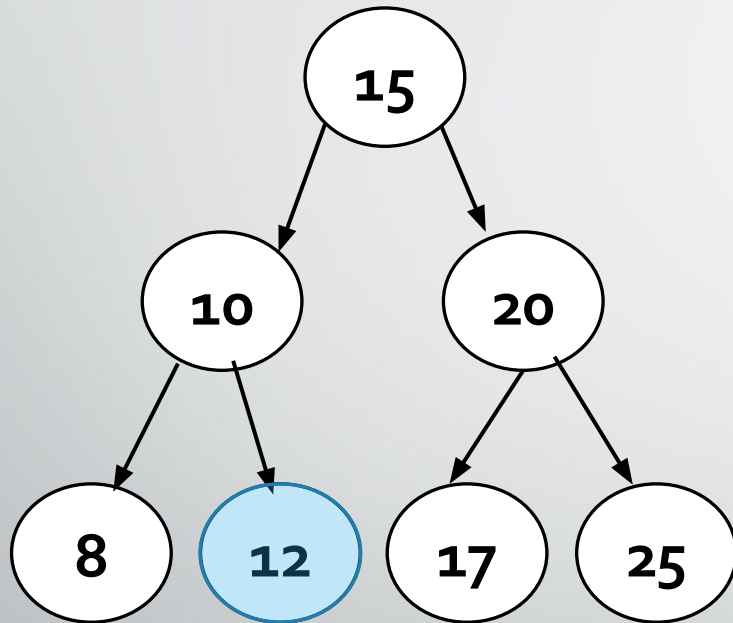


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+0+**0**+c(12)+c(20)

Count elements in the BST

✓ Ex: Count this tree

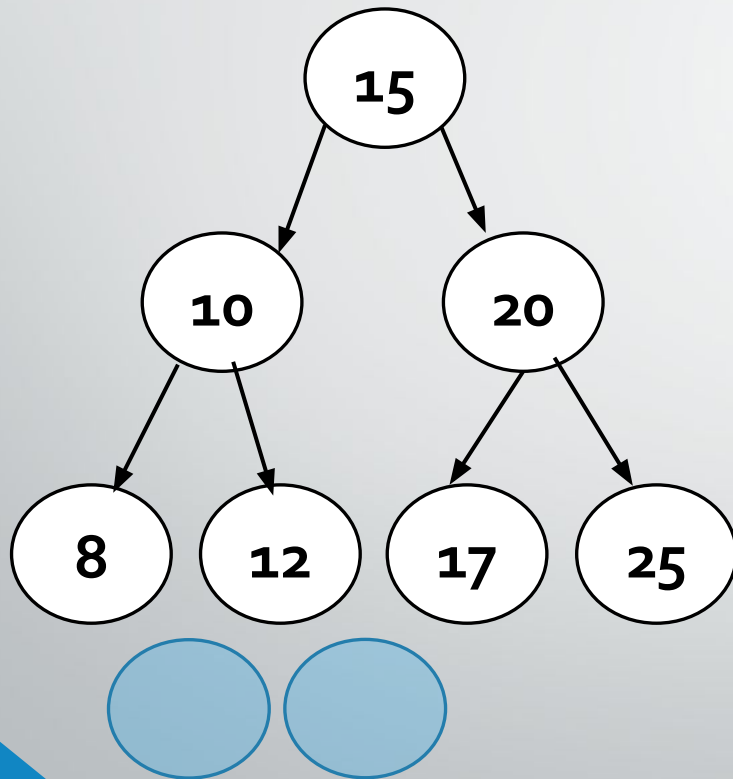


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+0+0+c(12)+c(20)
1+1+1+0+0+ **1+c(null)+c(null)** +c(20)

Count elements in the BST

✓ Ex: Count this tree

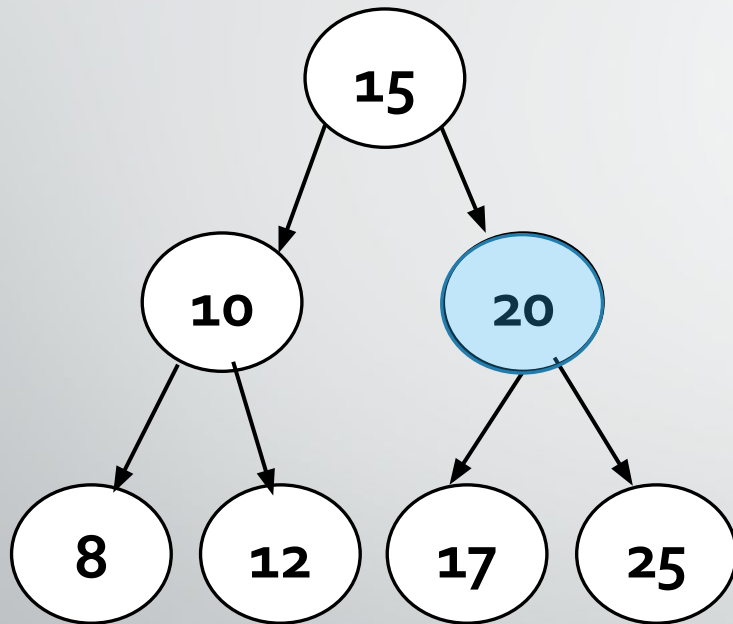


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+0+0+c(12)+c(20)
1+1+1+0+0+1+**0+0**+c(20)

Count elements in the BST

✓ Ex: Count this tree

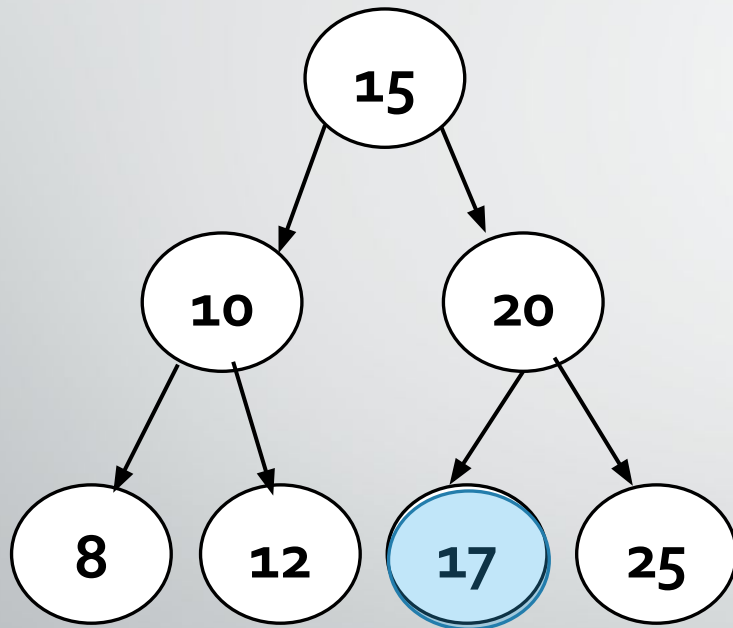


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+0+0+c(12)+c(20)
1+1+1+0+0+1+0+0+c(20)
1+1+1+0+0+1+0+0+ **1+c(17)+c(25)**

Count elements in the BST

✓ Ex: Count this tree

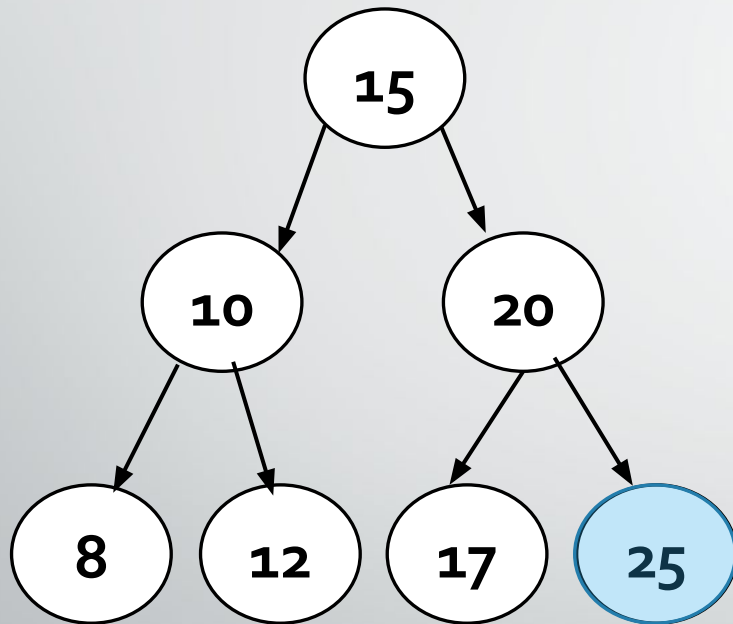


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+0+0+c(12)+c(20)
1+1+1+0+0+1+0+0+c(20)
1+1+1+0+0+1+0+0+1+c(17)+c(25)
1+1+1+0+0+1+0+0+1+**1+0+0**+c(25)

Count elements in the BST

✓ Ex: Count this tree

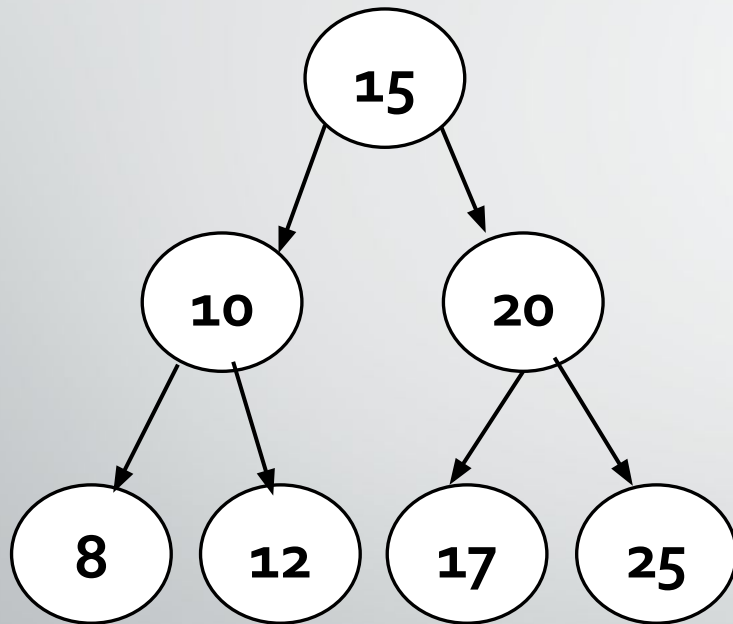


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+0+0+c(12)+c(20)
1+1+1+0+0+1+0+0+c(20)
1+1+1+0+0+1+0+0+1+c(17)+c(25)
1+1+1+0+0+1+0+0+1+1+0+0+ **1+0+0**

Count elements in the BST

✓ Ex: Count this tree

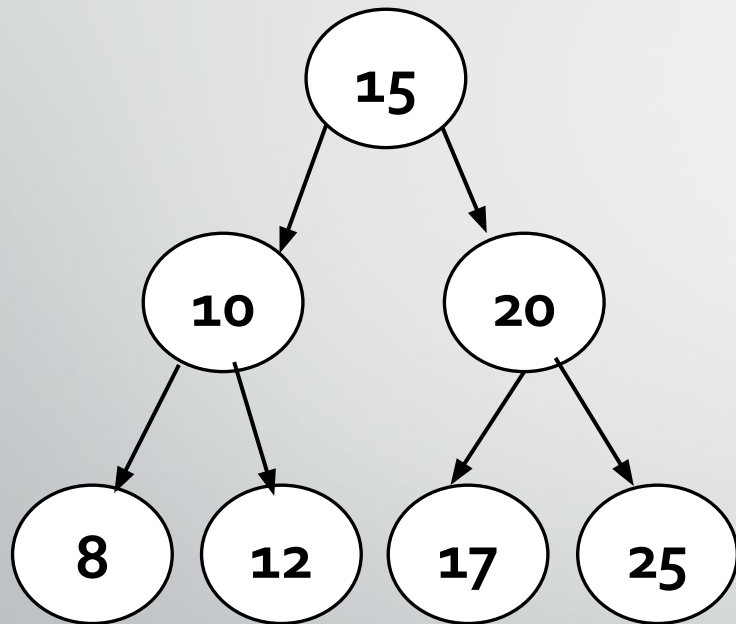


```
int count(Node *&root) {  
    if(root == NULL) {  
        return 0;  
    }  
    else {  
        return 1+count(root->left)+  
            count(root->right);  
    }  
}
```

count(15)
1+count(10)+count(20)
1+1+count(8)+count(12)+count(20)
1+1+1+c(null)+c(null)+c(12)+c(20)
1+1+1+0+0+c(12)+c(20)
1+1+1+0+0+1+0+0+c(20)
1+1+1+0+0+1+0+0+1+c(17)+c(25)
1+1+1+0+0+1+0+0+1+1+0+0+1+0+0 = 7

Find the Min element in the BST

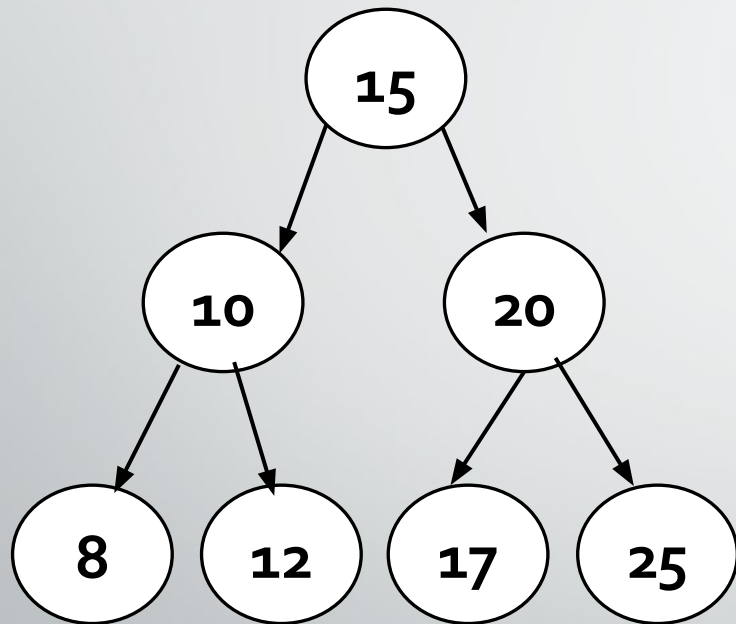
✓ How we can find the Min of a BST ?



Find the Min element in the BST

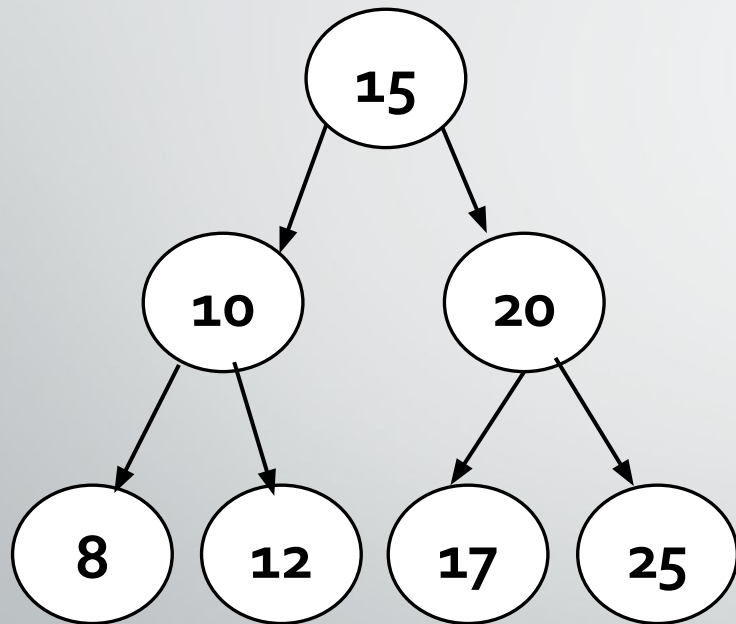
- ✓ Check if the tree is empty , then return not found or -1

```
int min(Node *&root) {  
    if (root == NULL) {  
        return -1;  
    }  
}
```



Find the Min element in the BST

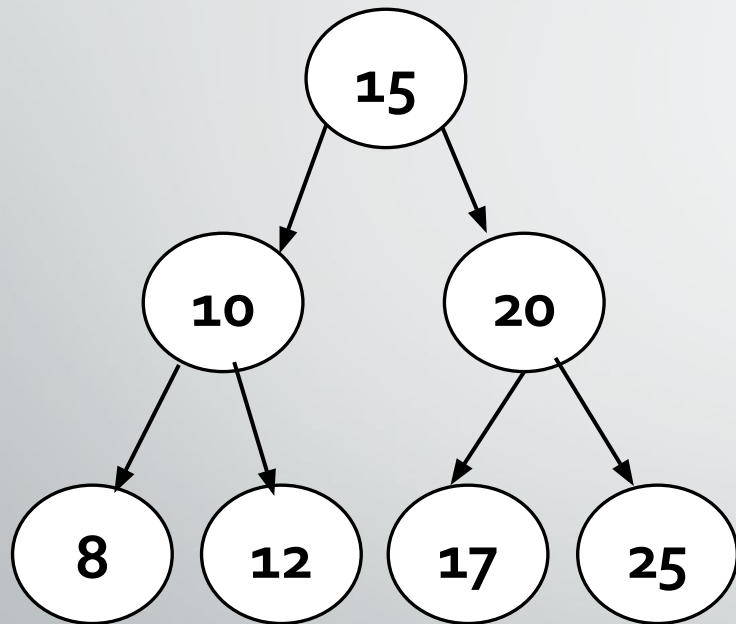
- ✓ Check if there is no left child , then you have found a min , else get the min of the left subtree



```
int min(Node *&root) {  
    if(root == NULL) {  
        return -1;  
    }  
    else if(root->left == NULL) {  
        return root->data;  
    }  
    else {  
        return min(root->left);  
    }  
}
```

Find the Min element in the BST

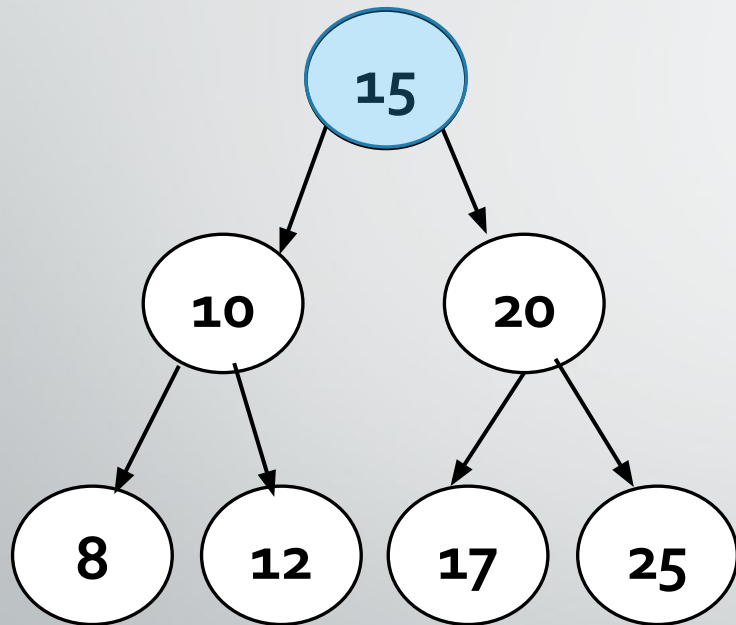
✓ Ex: Find the Min



```
int min(Node *&root) {  
    if(root == NULL) {  
        return -1;  
    }  
    else if(root->left == NULL) {  
        return root->data;  
    }  
    else {  
        return min(root->left);  
    }  
}
```

Find the Min element in the BST

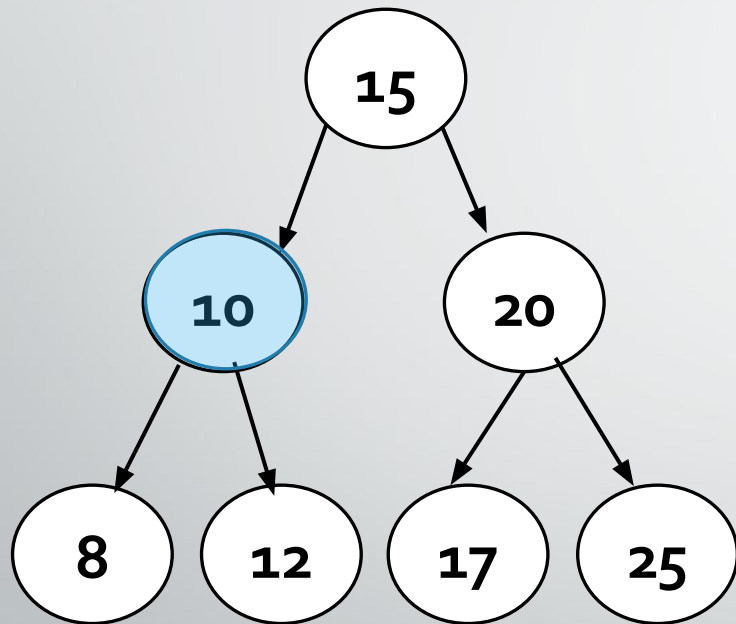
✓ Ex: Find the Min



```
int min(Node *&root) {  
    if(root == NULL) {  
        return -1;  
    }  
    else if(root->left == NULL) {  
        return root->data;  
    }  
    else {  
        return min(root->left);  
    }  
}
```

Find the Min element in the BST

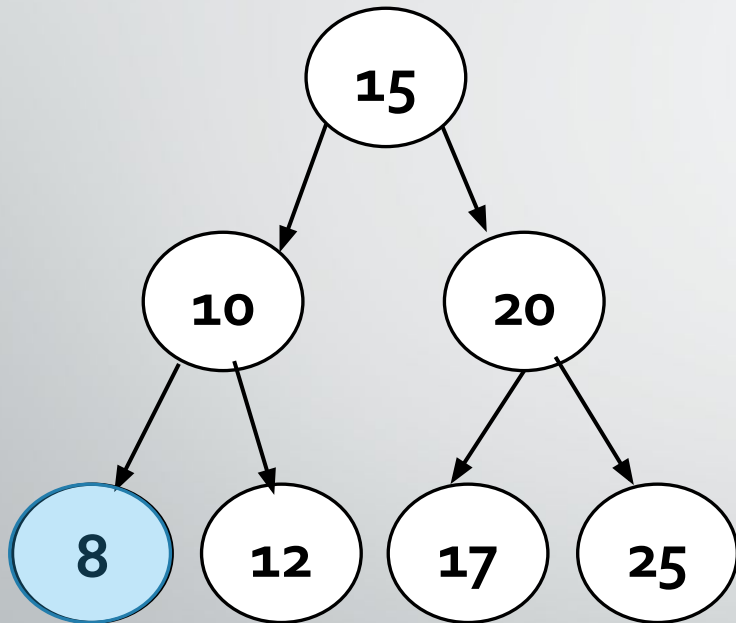
✓ Ex: Find the Min



```
int min(Node *&root) {  
    if(root == NULL) {  
        return -1;  
    }  
    else if(root->left == NULL) {  
        return root->data;  
    }  
    else {  
        return min(root->left);  
    }  
}
```

Find the Min element in the BST

✓ Ex: Find the Min



```
int min(Node *&root) {  
    if(root == NULL) {  
        return -1;  
    }  
    else if(root->left == NULL) {  
        return root->data;  
    }  
    else {  
        return min(root->left);  
    }  
}
```

✓ Return 8

HW4: Write C++ or Pseudo Code to:

- ✓ Create functions for the following :
sum , max
- ✓ Calculate the Depth of a Node
- ✓ Copy a Binary Search Tree
- ✓ Check if 2 Trees are equal

END of Section

Thank You

